

实验八、Xilinx Microblaze Design

介绍:

本实验介绍如何添加一个 Microblaze 的软核，并添加 1 个 GPIO 的接口 IP，以连接板卡上的 LED；同时添加 uartlite 的 ip 核，以连接板卡上的串口，以超级终端发送的 ascii 码转换成 2 进制，在 led 上显示。

目标:

当完成本实验后，会学习到:

用 vivado 实现模块化设计

在 block design 界面，添加 microblaze 核

添加 GPIO IP 核

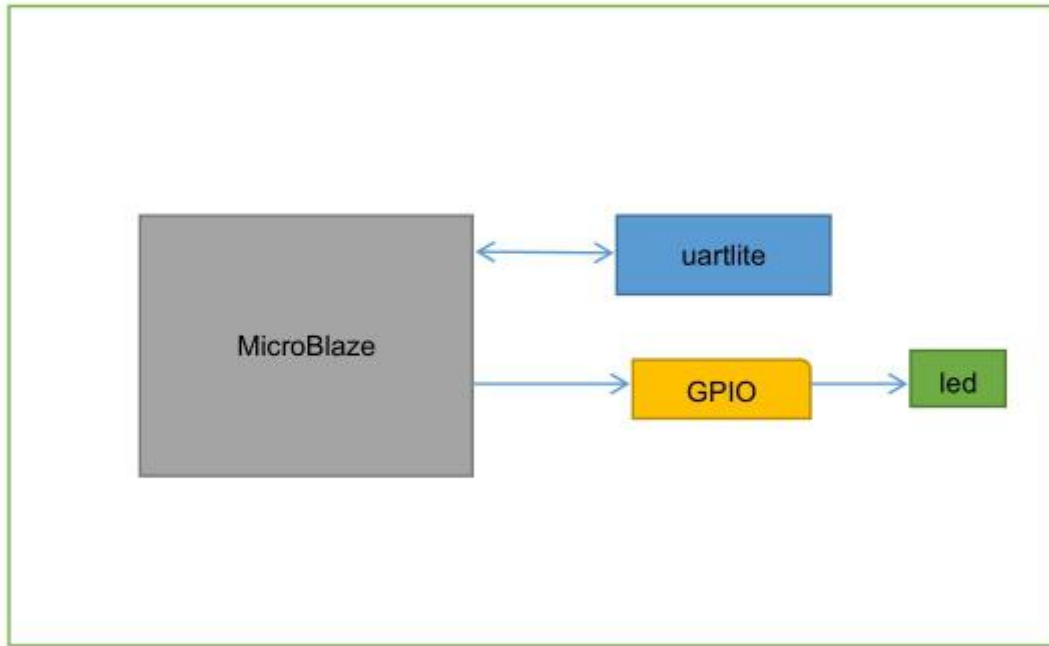
添加串口 IP 核

导出硬件信息，并在 SDK 界面上实现 microblaze 的应用程序开发

实验流程:



设计框图:

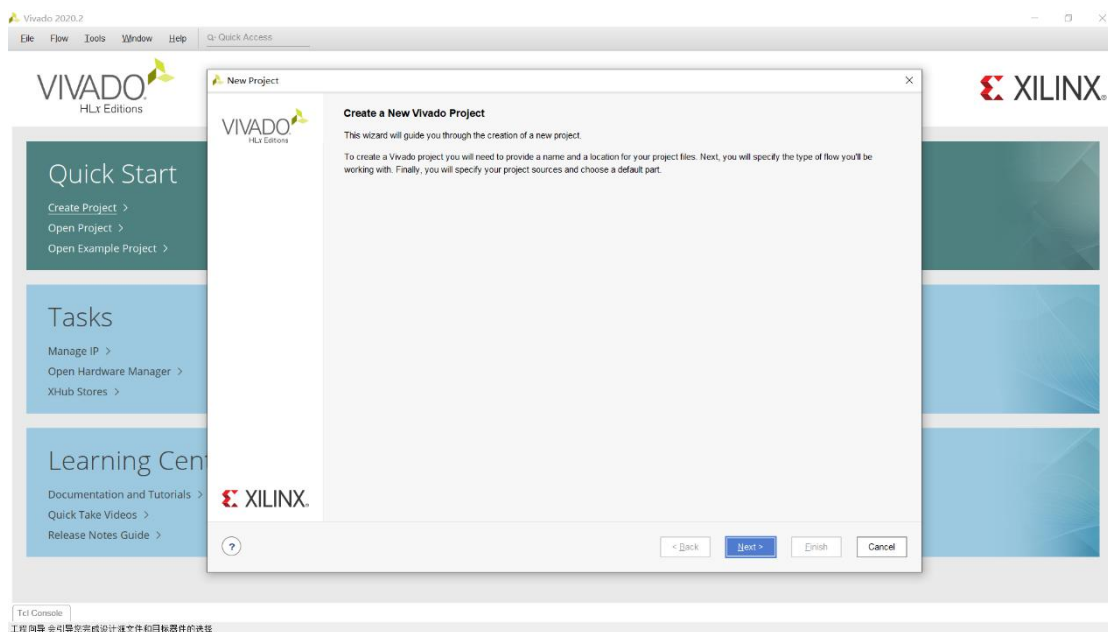


实验步骤:

1. 打开 VIVADO 工具，并创建一个全新的设计，点击 create new project。

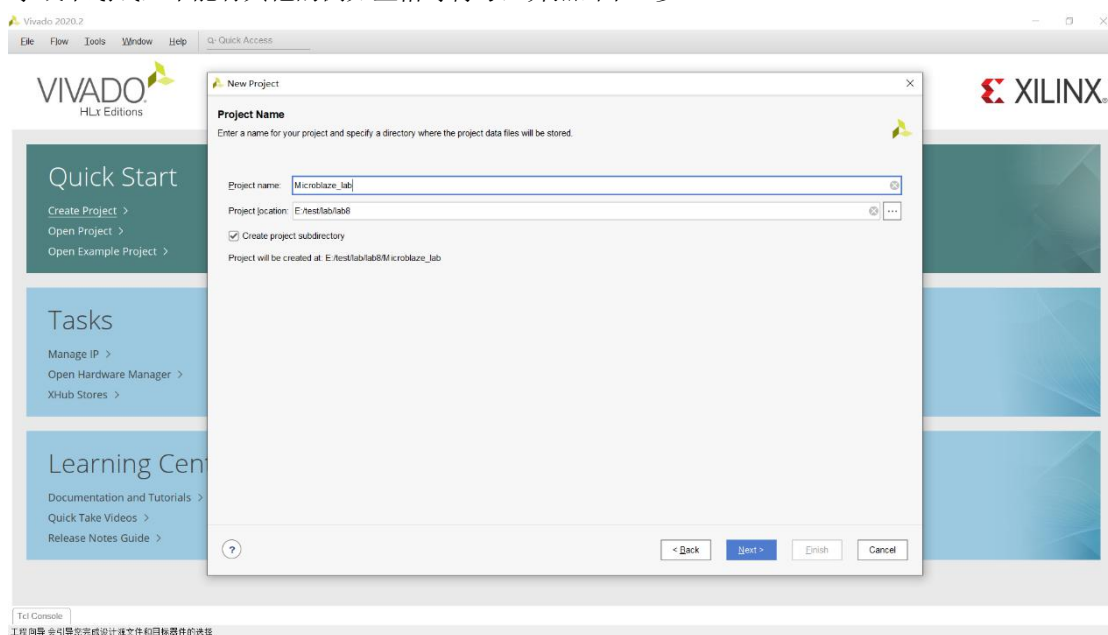


2. 在如下界面点击 next。



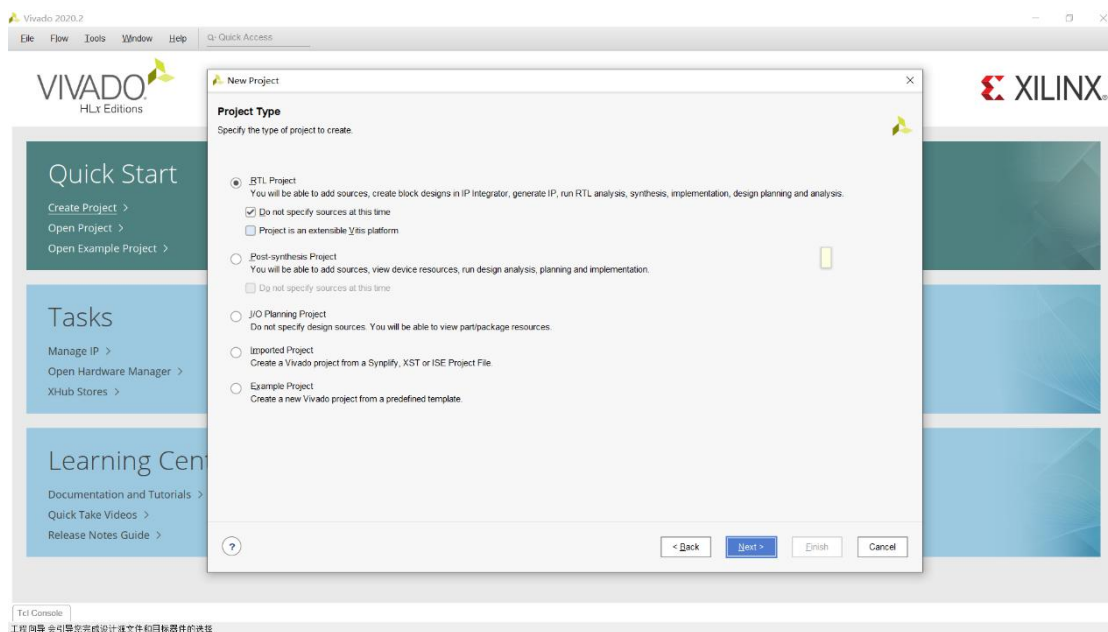
Tcl Console
工程向导 会引导您完成设计文件和目标器件的选择

3. 给出工程的名称与存放的位置，需注意存放位置需是英文字母开头，后面可以跟数字或下划线，不能有其他的例如空格等符号，并点击下一步。

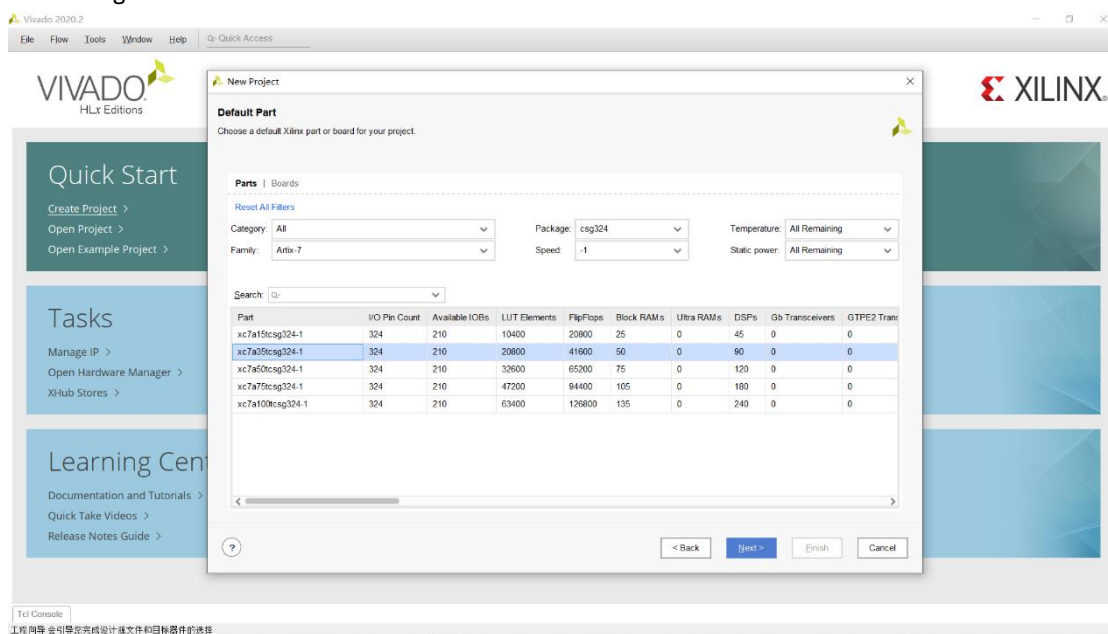


Tcl Console
工程向导 会引导您完成设计文件和目标器件的选择

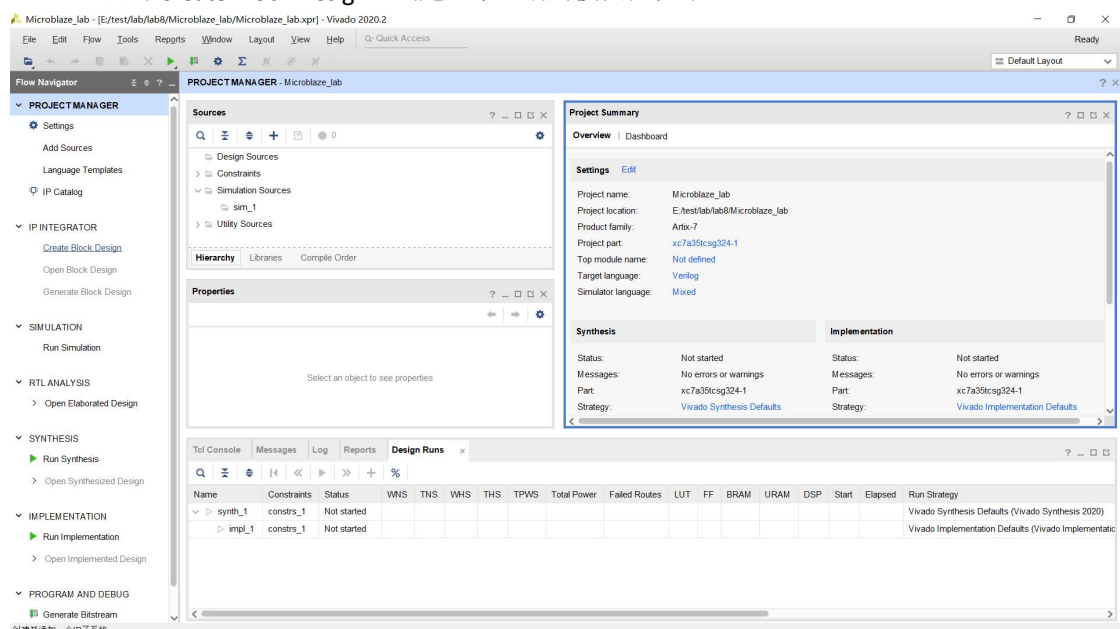
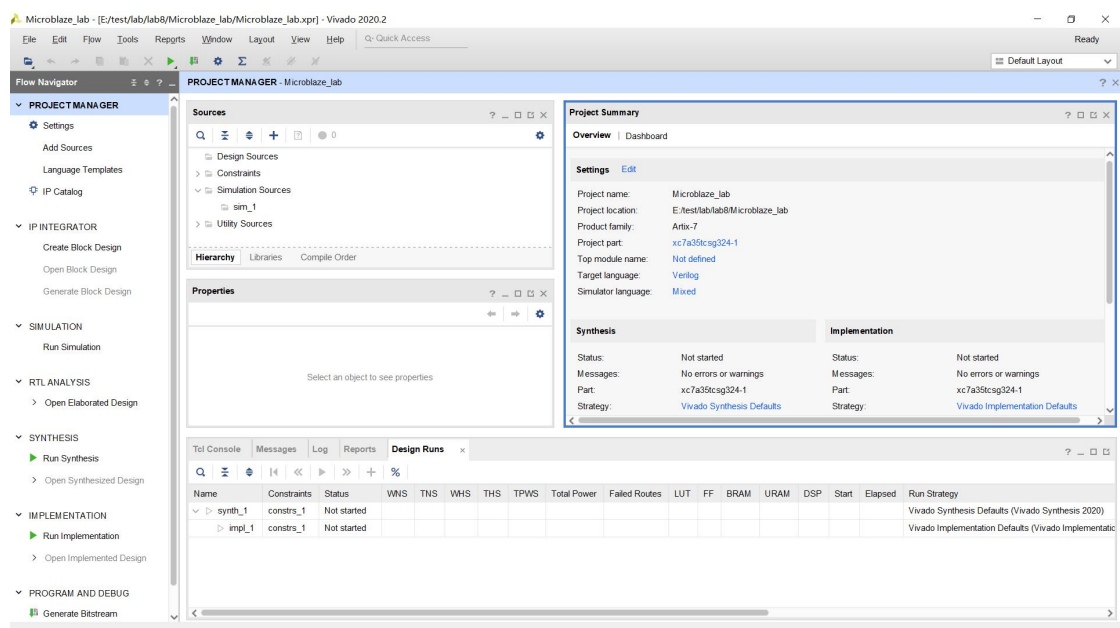
4. 选择 RTL 设计，并勾选 Do not specify sources at this time。在现阶段不添加源文件。并点击 Next 进入下一步。

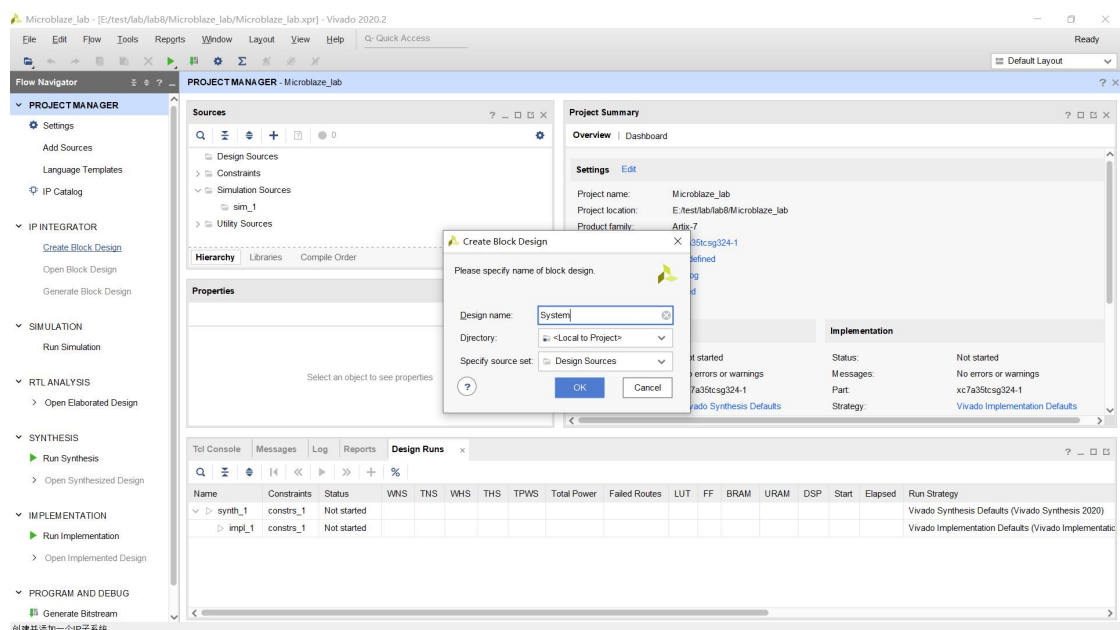


5. 选择芯片类型，本实验以 E-Elements 专为竞赛设计的 Artix-7 核心板为例，因此选择 xc7a35tcsg324-1，并点击 Next->Finish。

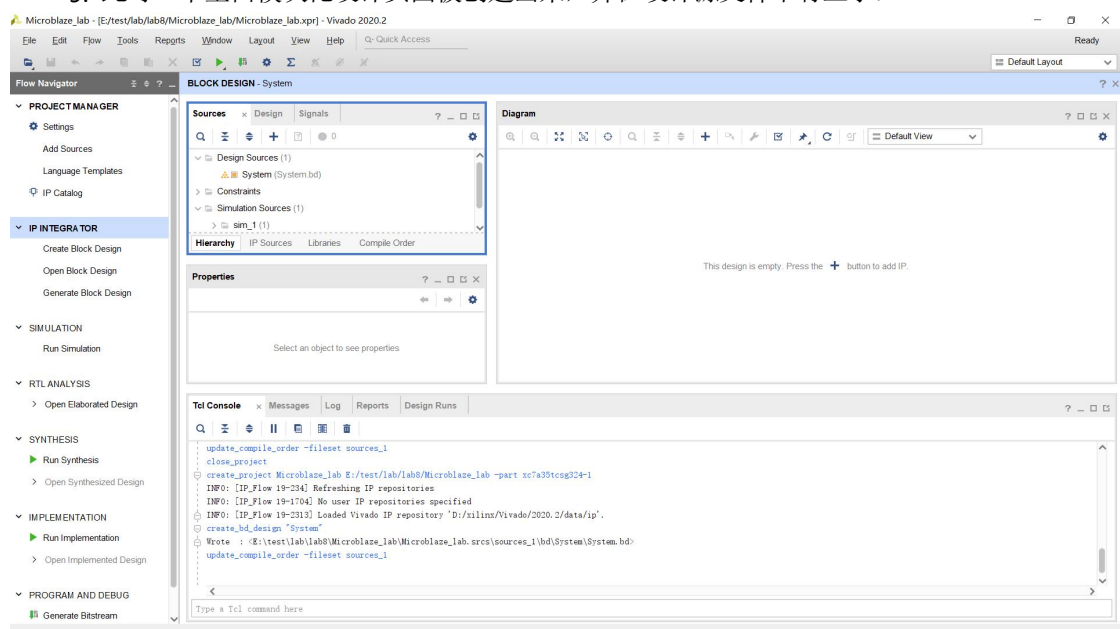


6. 建立好全新的工程:

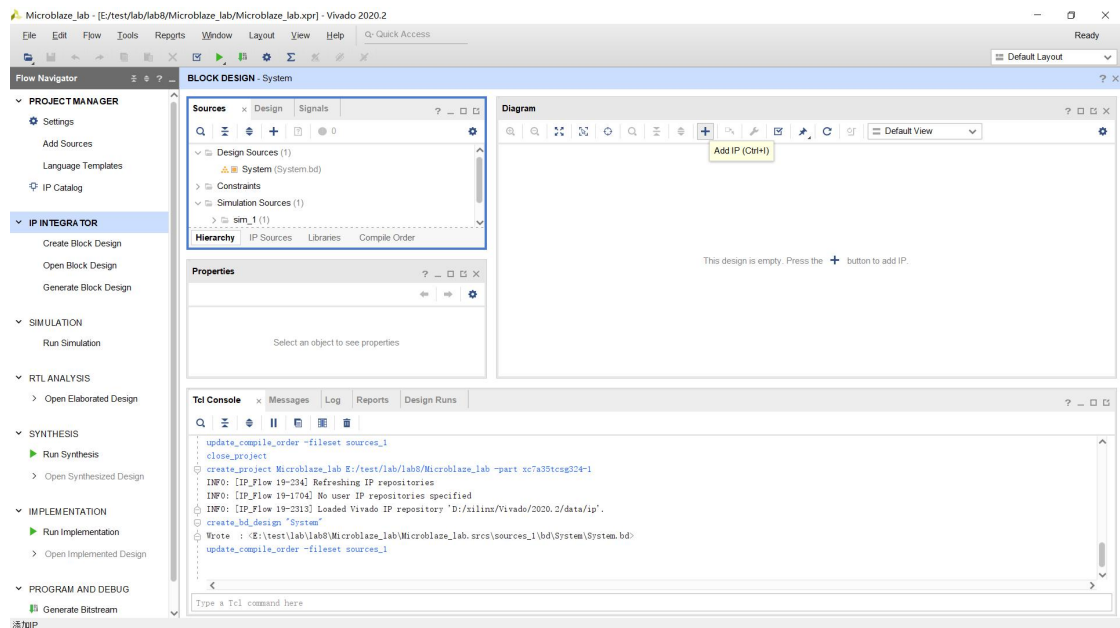




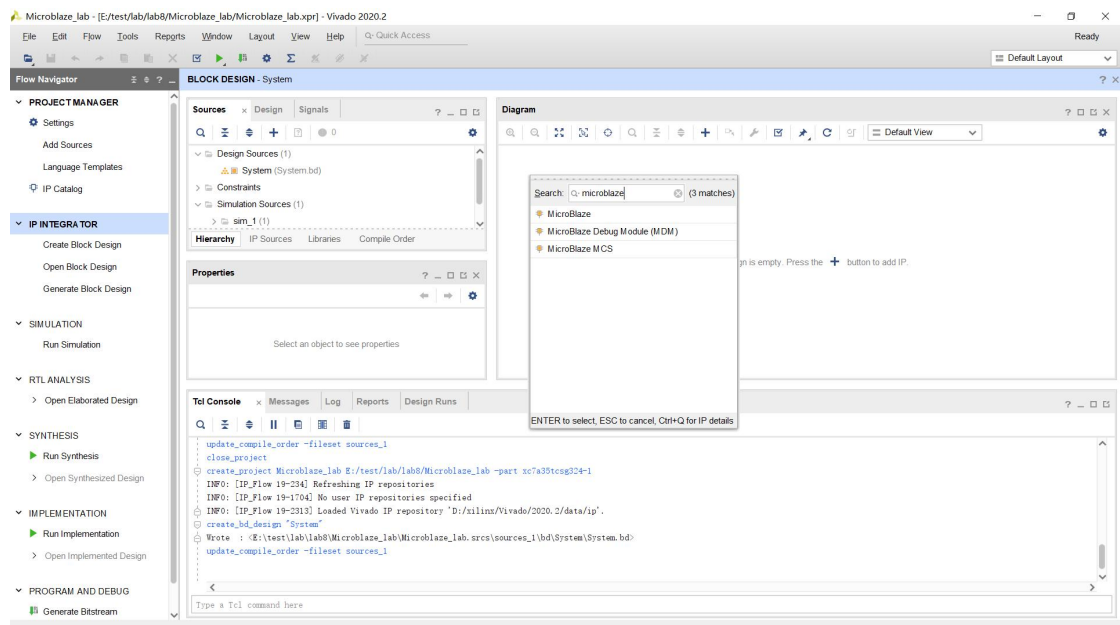
9. 此时一个空白模块化设计页面被创建出来，并在设计源文件中有显示：



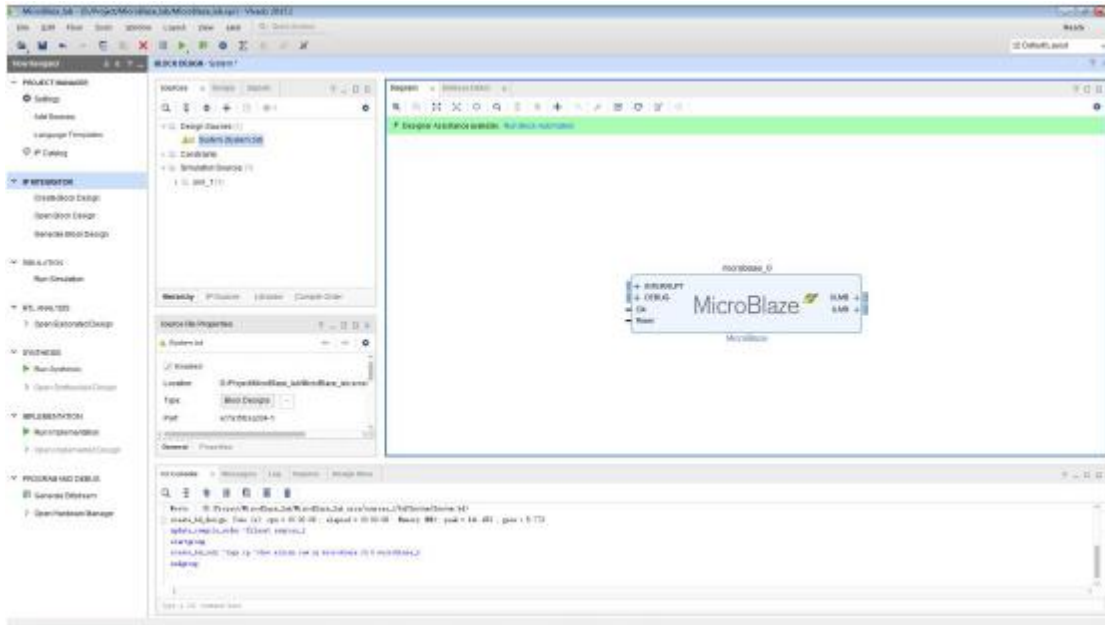
10. 点击 Add IP，或在模块化设计页面空白处点击右键，选择 Add IP。



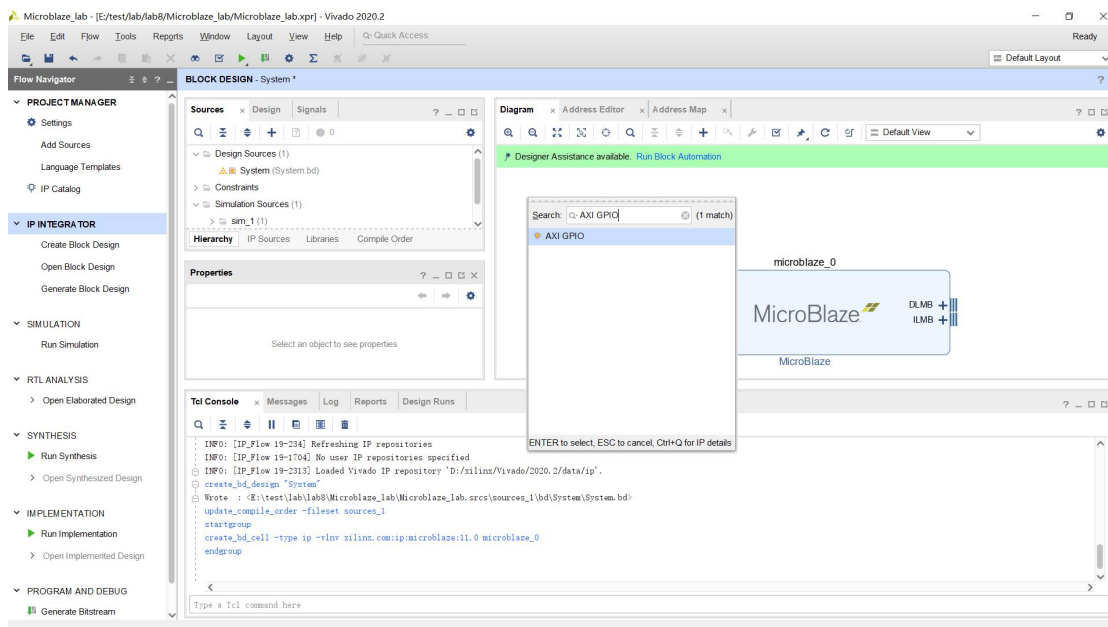
11. 搜索 MicroBlaze 并双击添加:

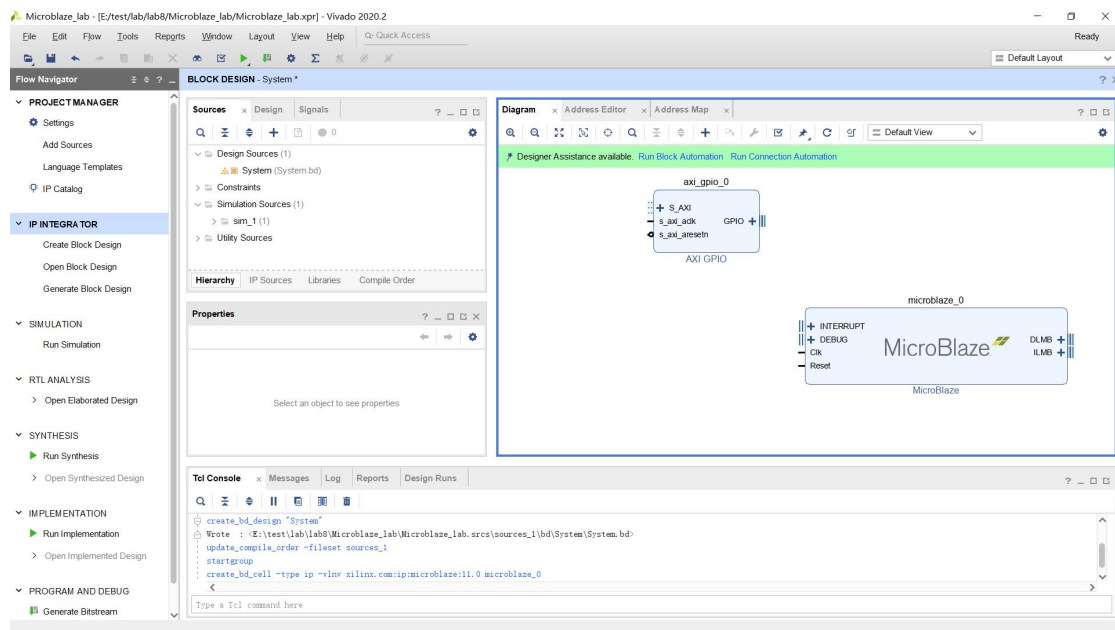


12. 可见 MicroBlaze 被添加到 Block Design 界面。

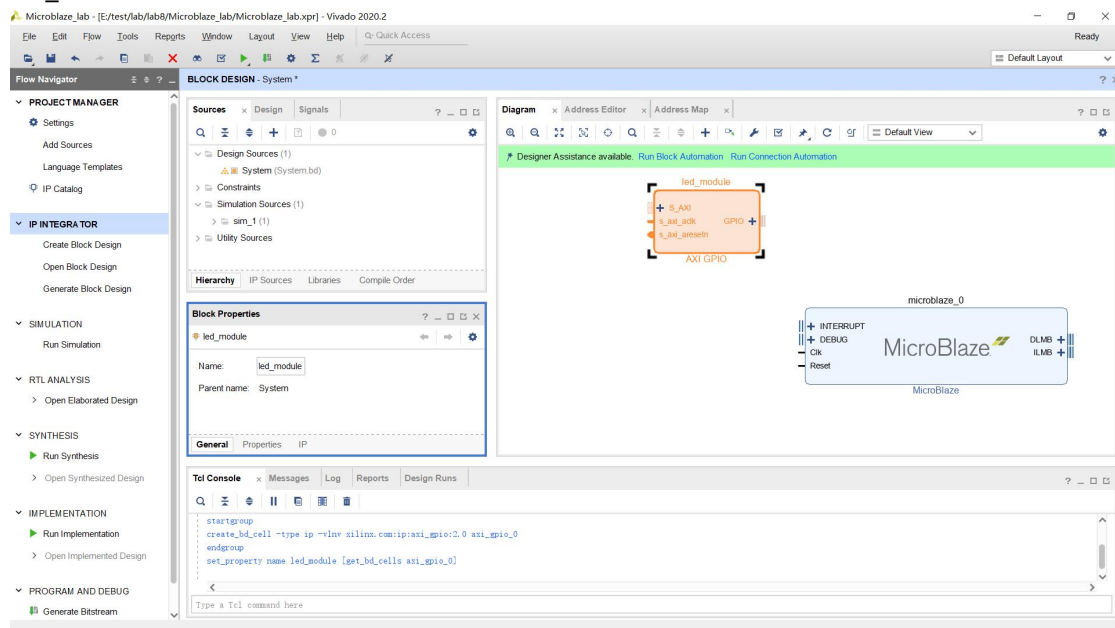


13. 同上方法添加 AXI GPIO IP:

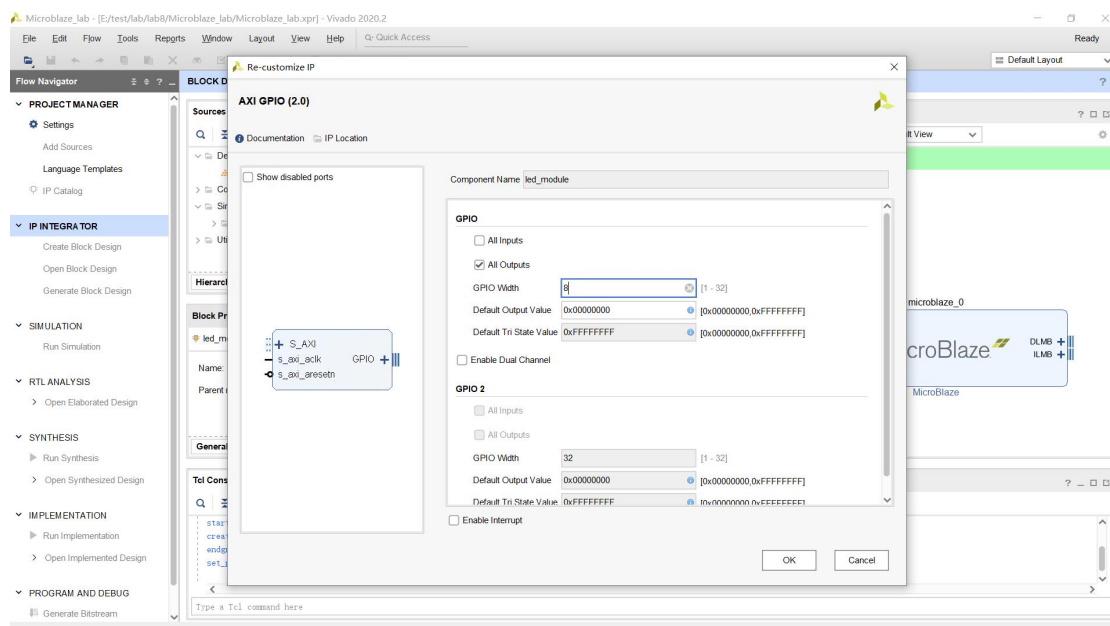




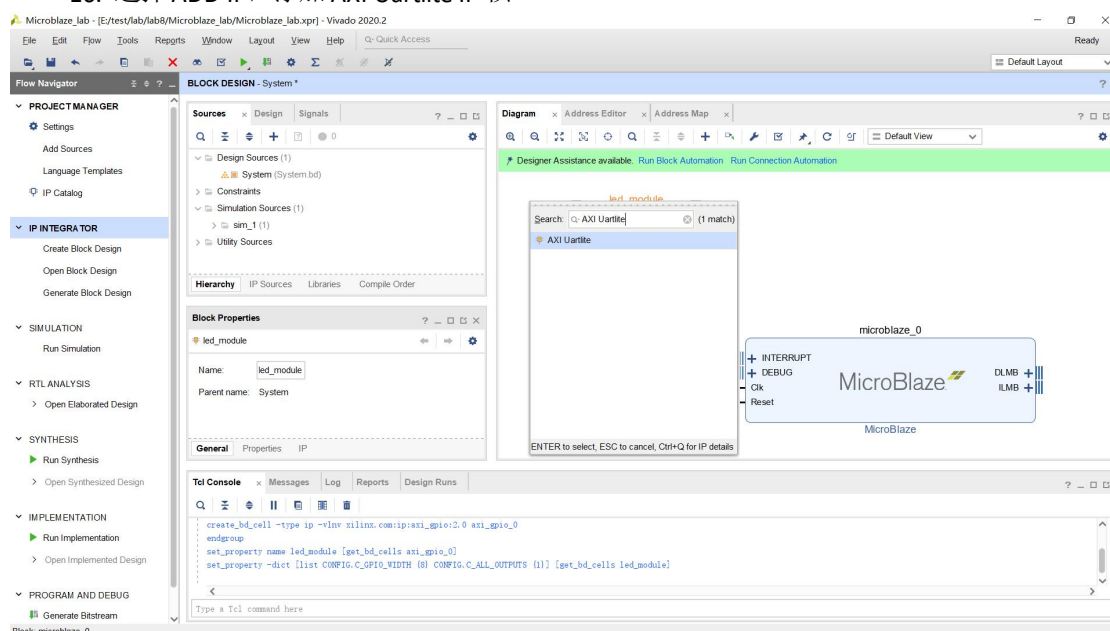
14. 点击 `axi_gpio_0`，在 Block Properties 窗口可以更改 IP 的名称，将名称改为 `led_module`。

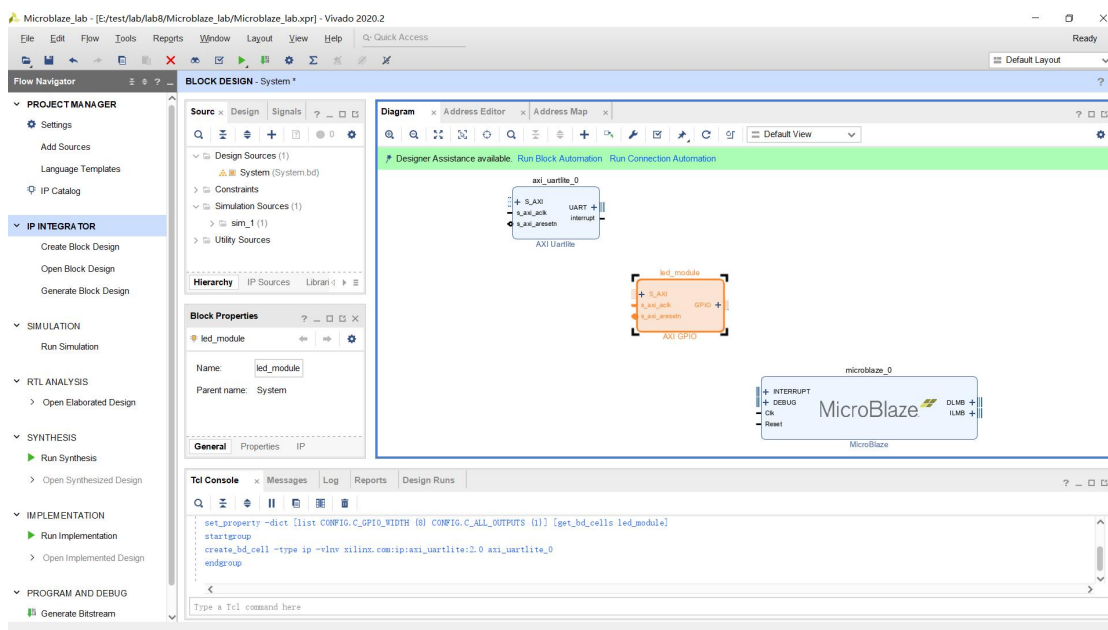


15. 双击 AXI GPIO 模块，因为作为 led 输出，所以勾选 `all outputs`，在 GPIO 位宽改为 8bits，作为 8 位 led 输出：

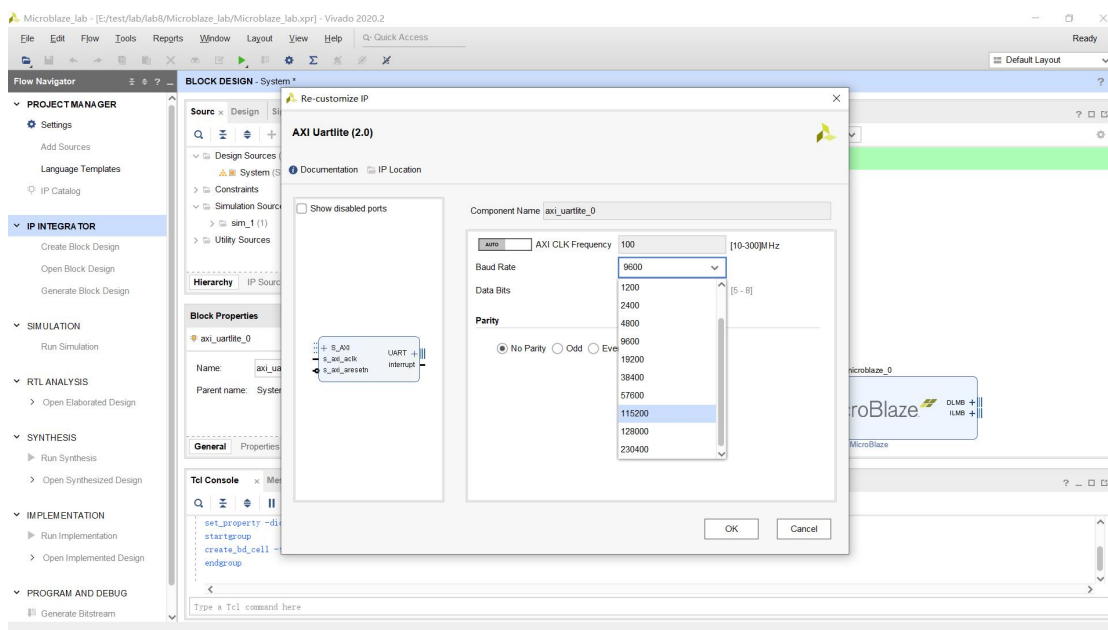


16. 选择 ADD IP, 添加 AXI Uartlite IP 核。

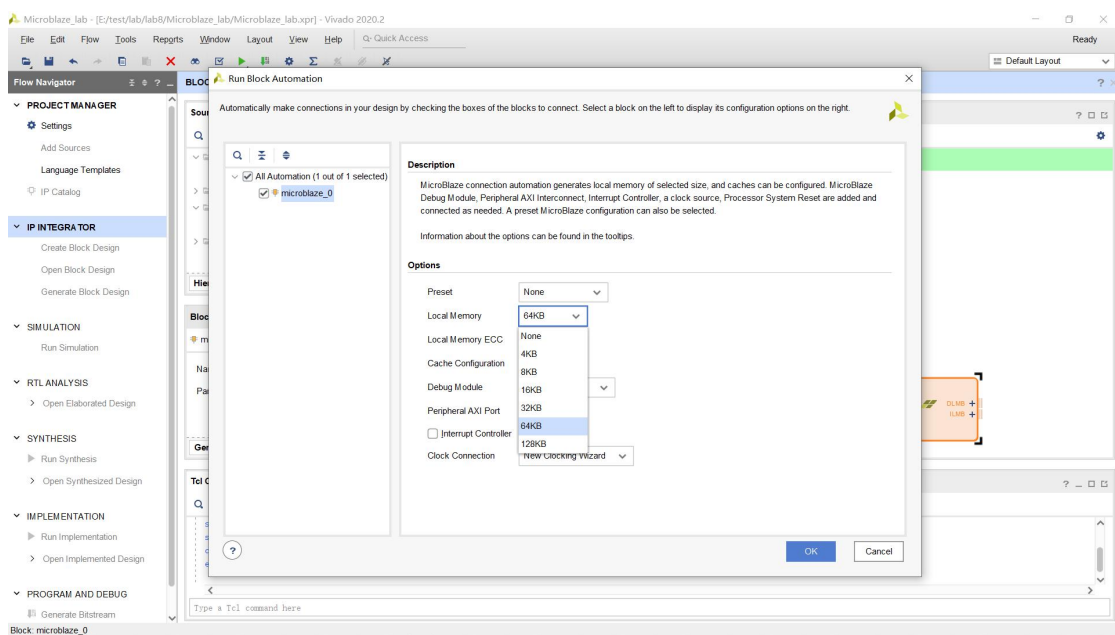
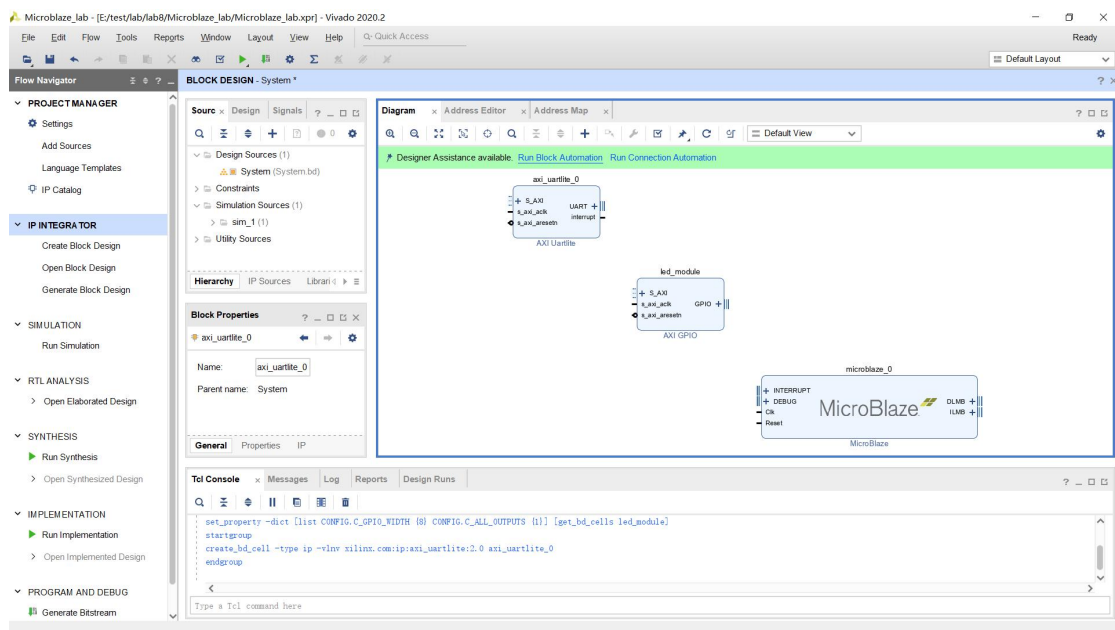


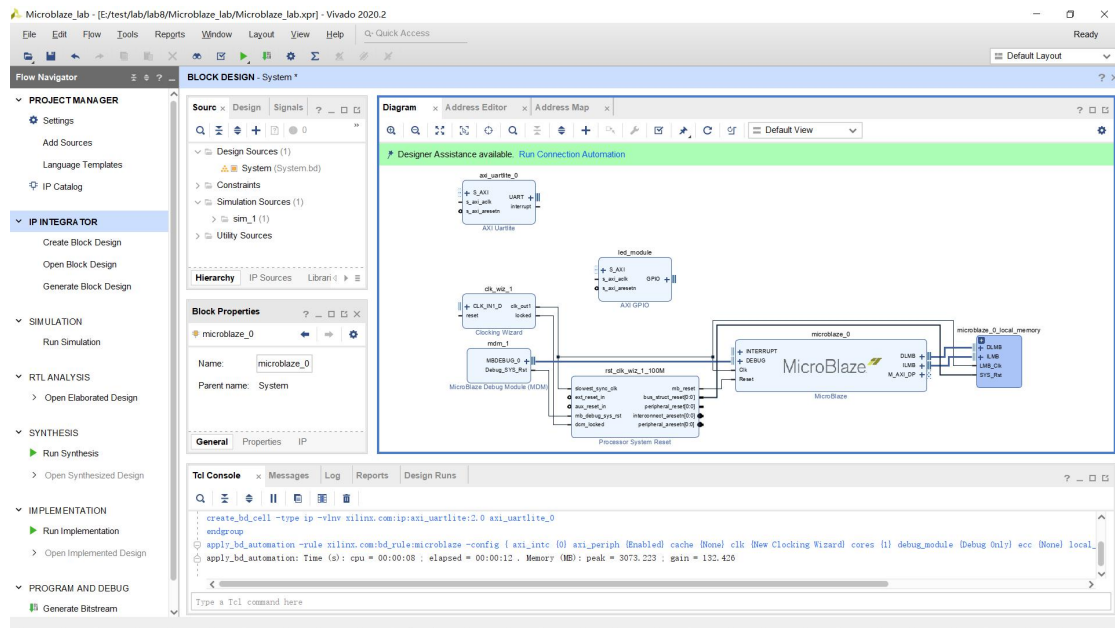


17. 双击 axi_uartlite_0，修改波特率为 115200（此处为串口波特率，可保持默认，亦可自己修改），并点击 OK。

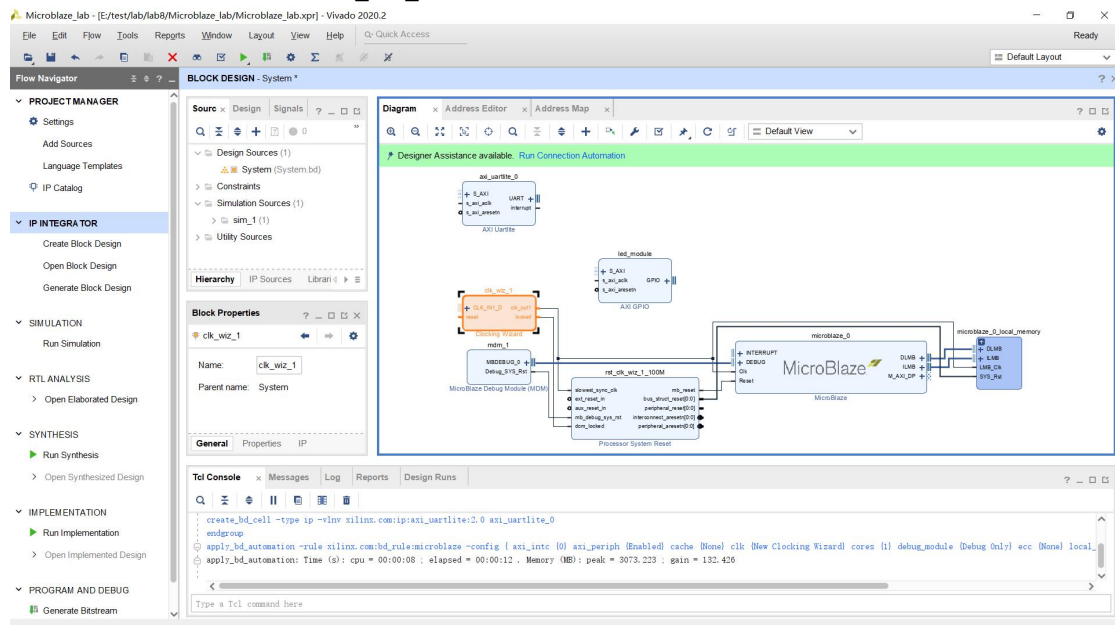


18. 点击 Run Block Automation，软件自动生成基于本系统需求的 IP，例如时钟模块、复位模块、总线接口等。Local Memory 选择 64KB，点击 OK。

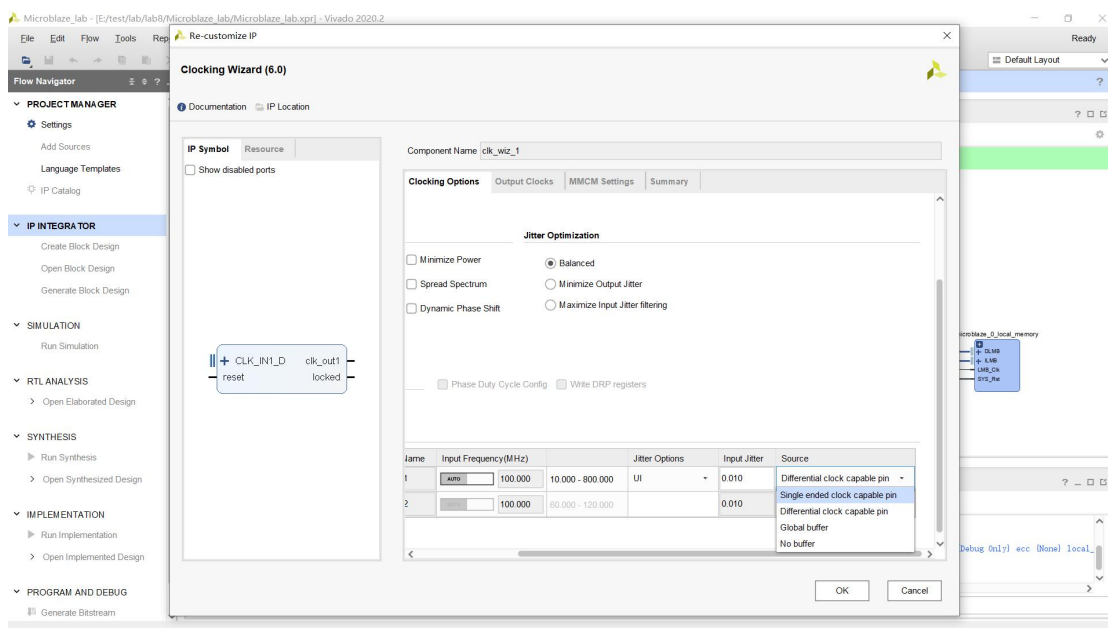




19. 选择时钟管理模块 clk_wiz_1，并双击。

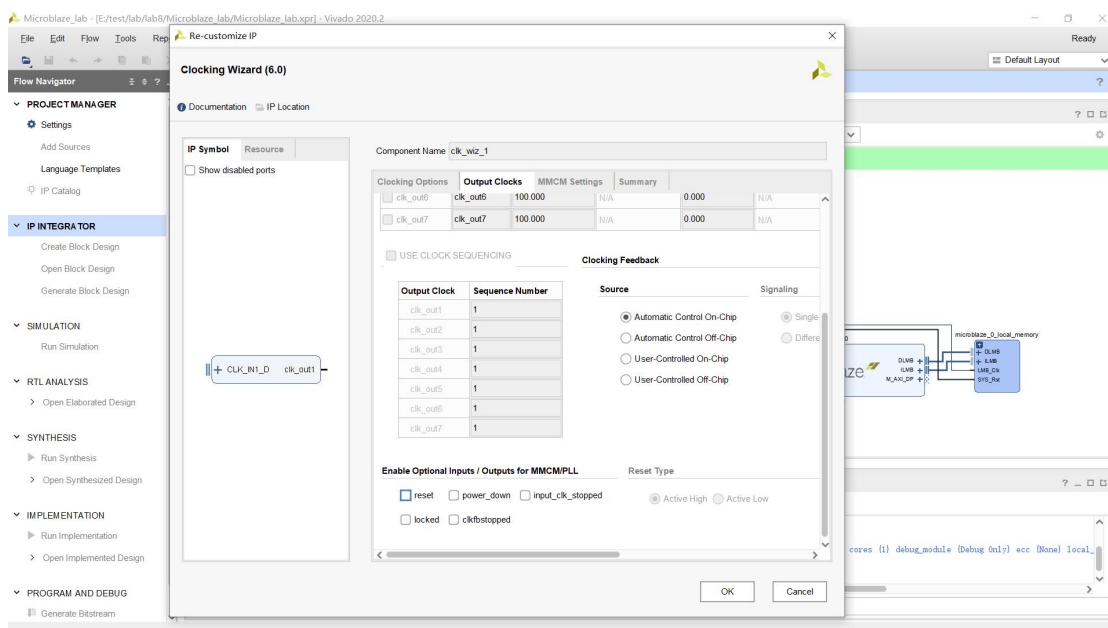


20. 把下方输入时钟源改为单端输入 Single ended clock capable。

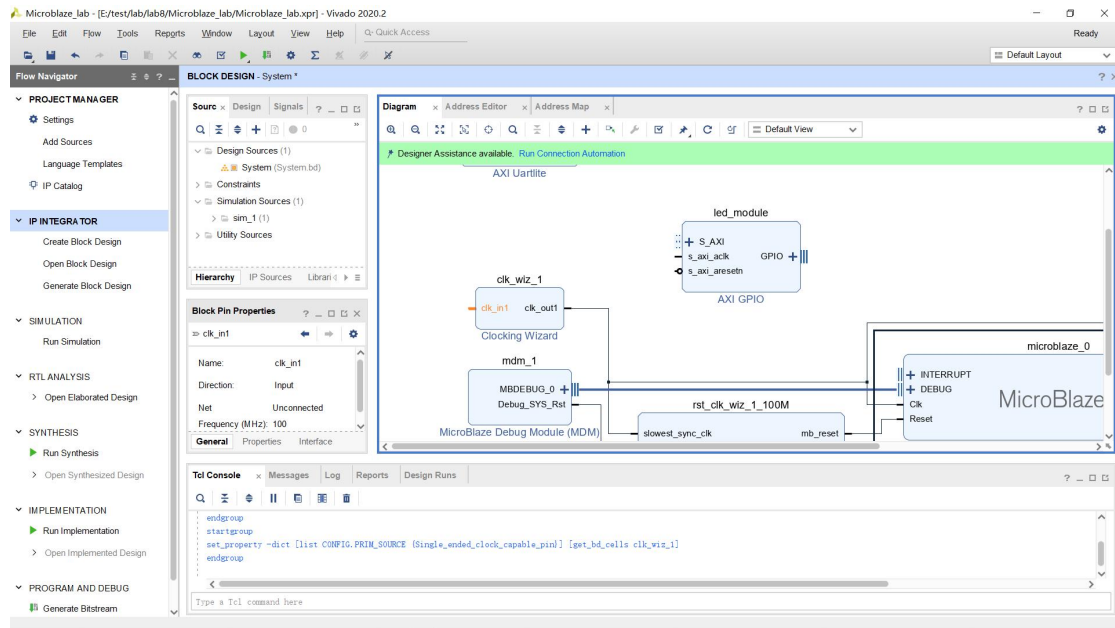


21. 输出时钟依然保持 100, 并把窗口拉到最下面, 把 reset、locked 的勾去掉。点击

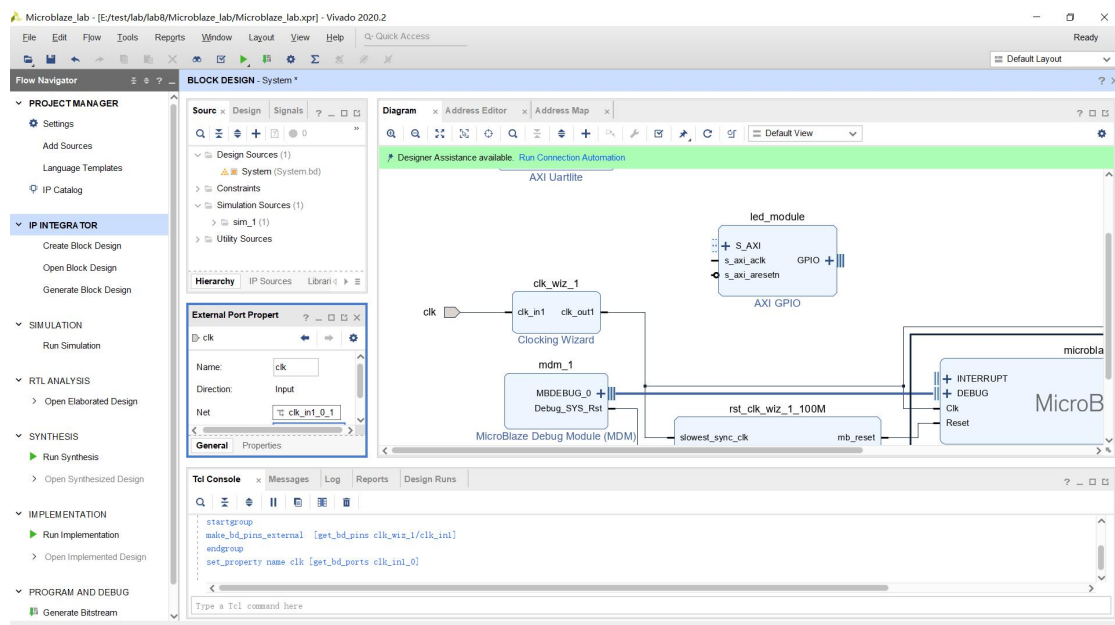
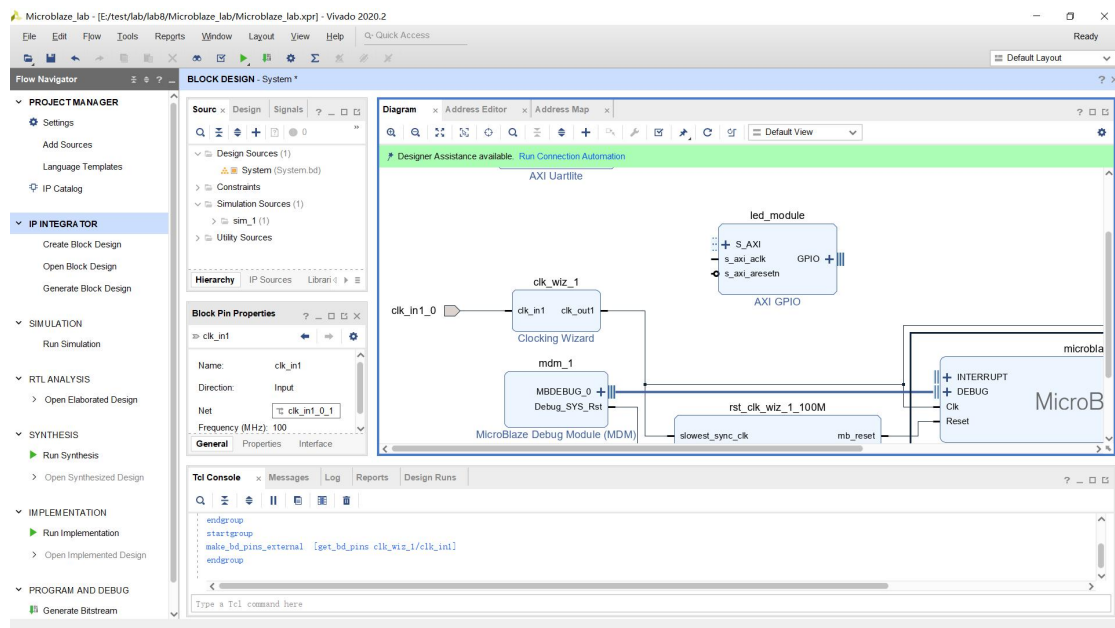
OK。

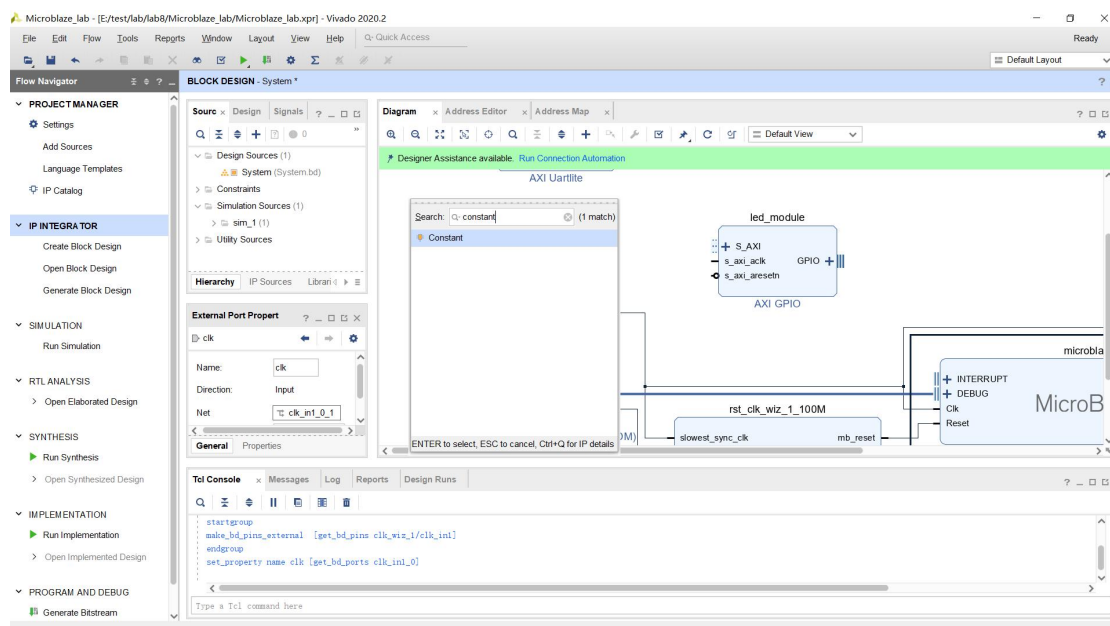


22. 可以看到时钟模块被更新, 选择 clk_in1 管脚让其变成黄色, 对管脚点击右键, 选择 Make External。

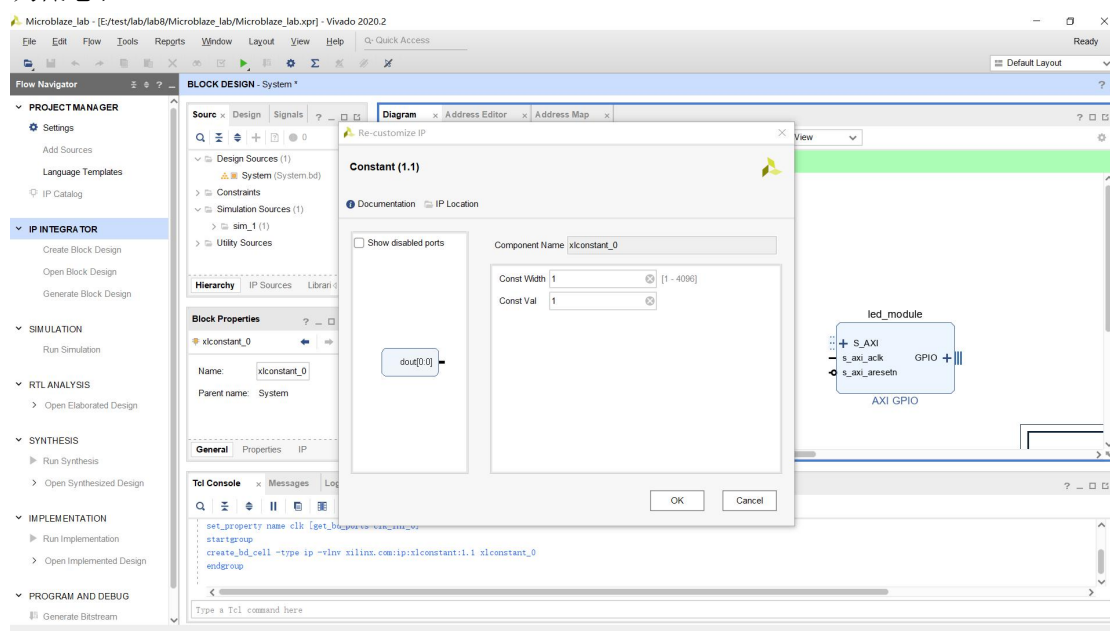


23. 时钟输入管脚被引出来:

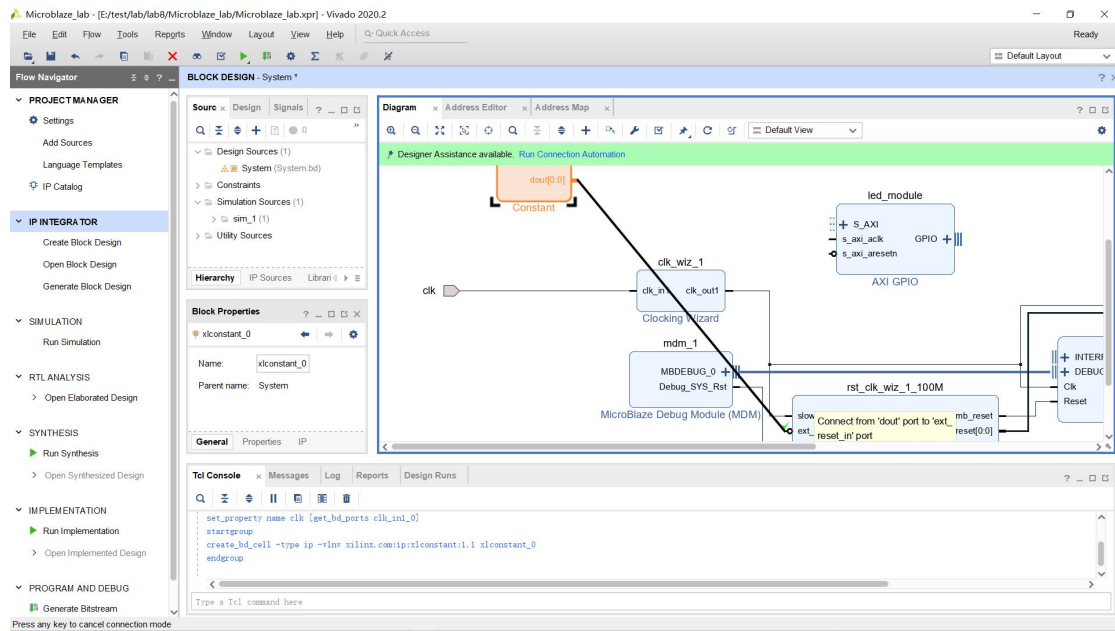




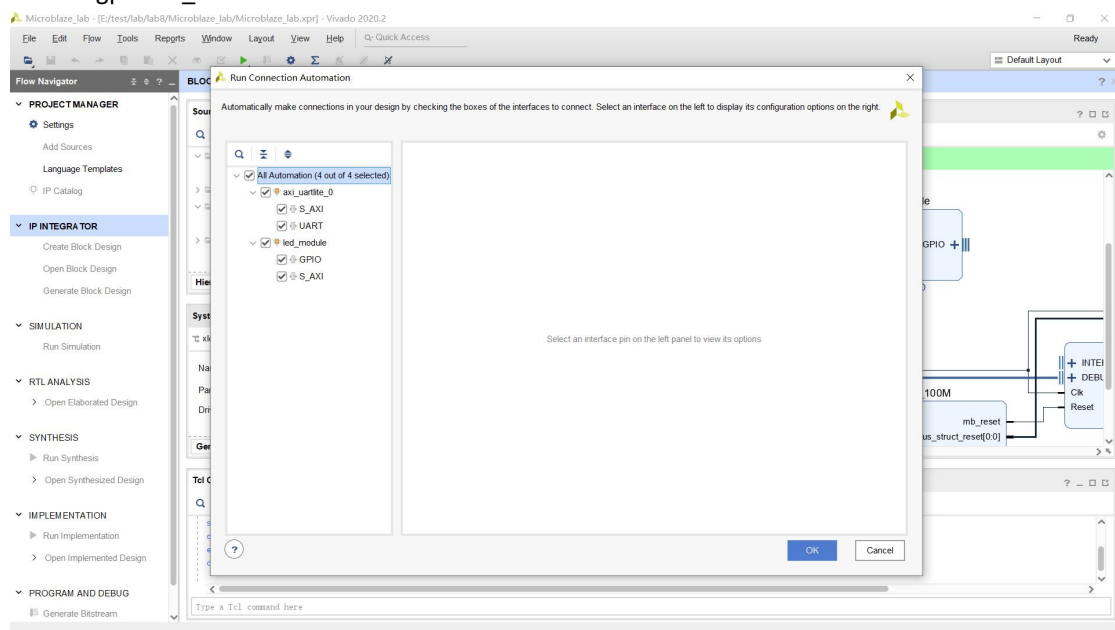
2) 双击常量 IP, 把输出设置为 1 位, 常量值为 1, 因为为了保持不复位状态, MB 复位为低电平。

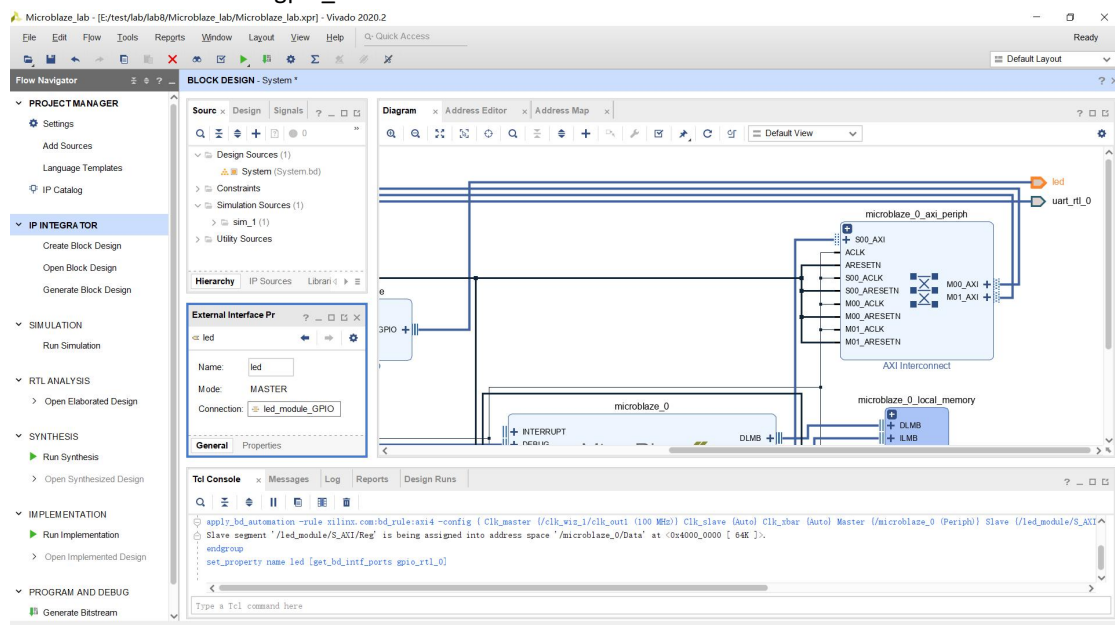
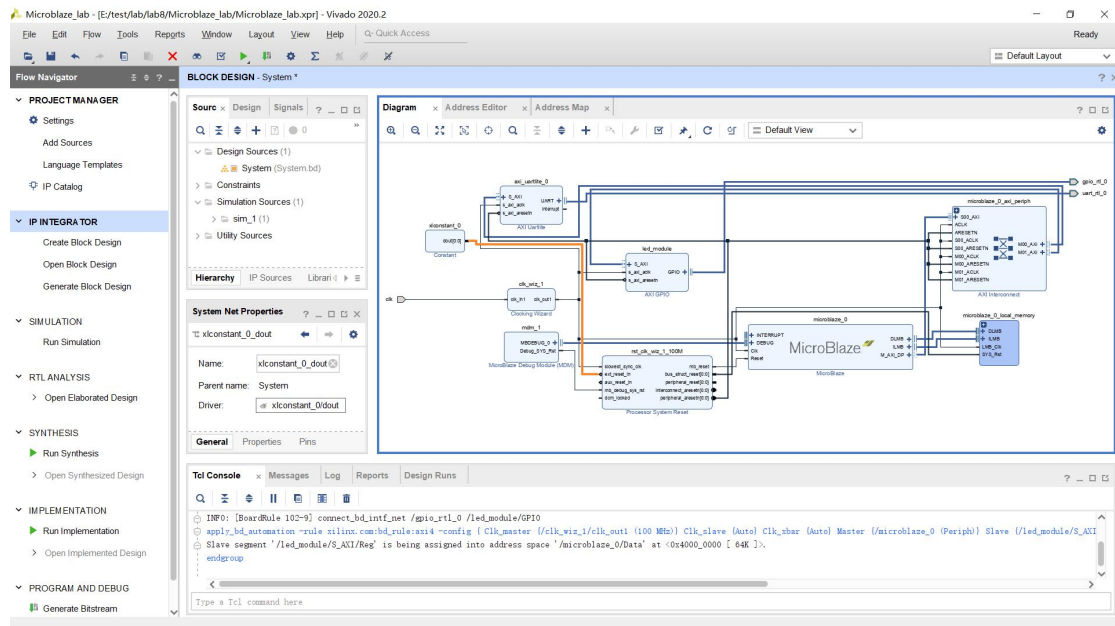


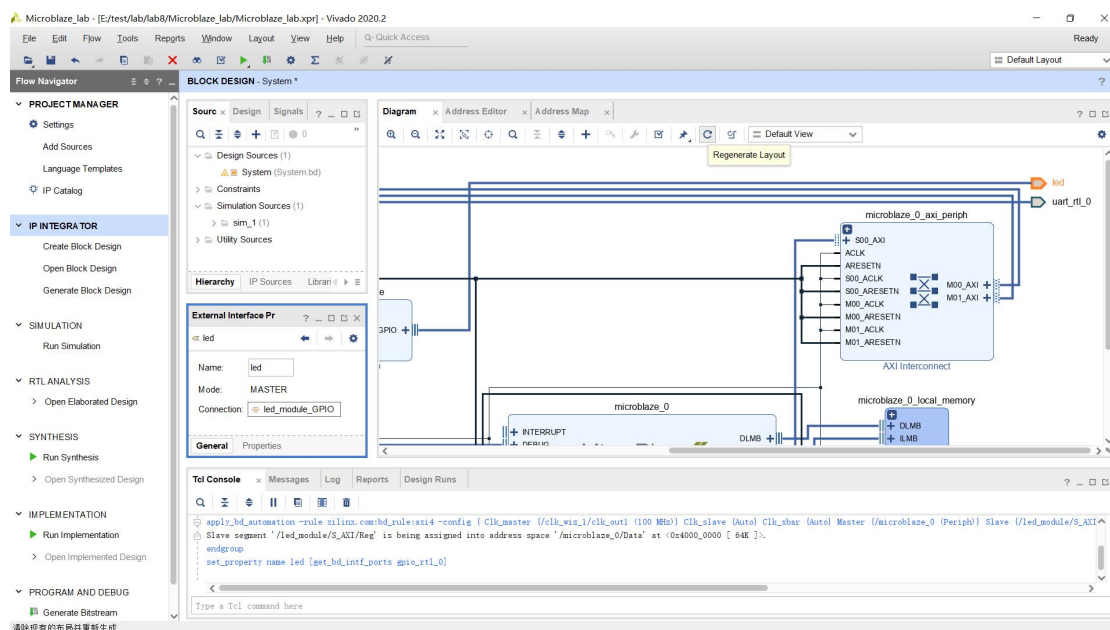
3) 把输出与 ext_reset_in 进行相连, 相连方法, 点击 dout 的端口, 按住左键, 出现铅笔标志, 进行拖线, 连到 ext_reset_in 接口。



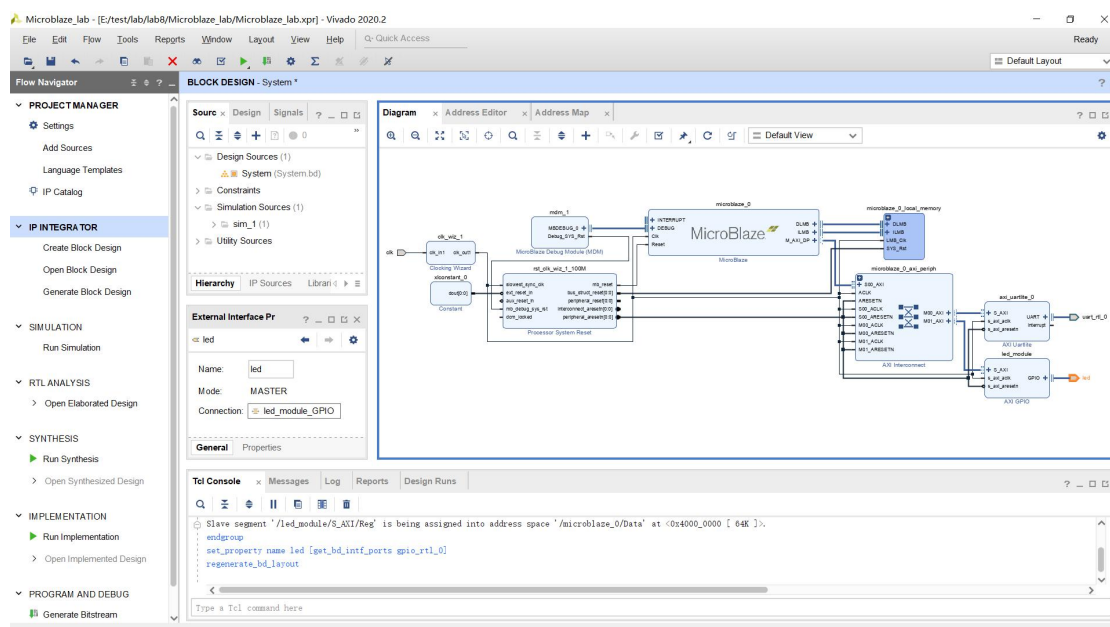
26. 点击上方 Run Connection Automation，并勾选上 All Automation，并点击 OK。软件自动把 gpio led_module 和 uartlite 与 MicroBlaze 核进行相连。



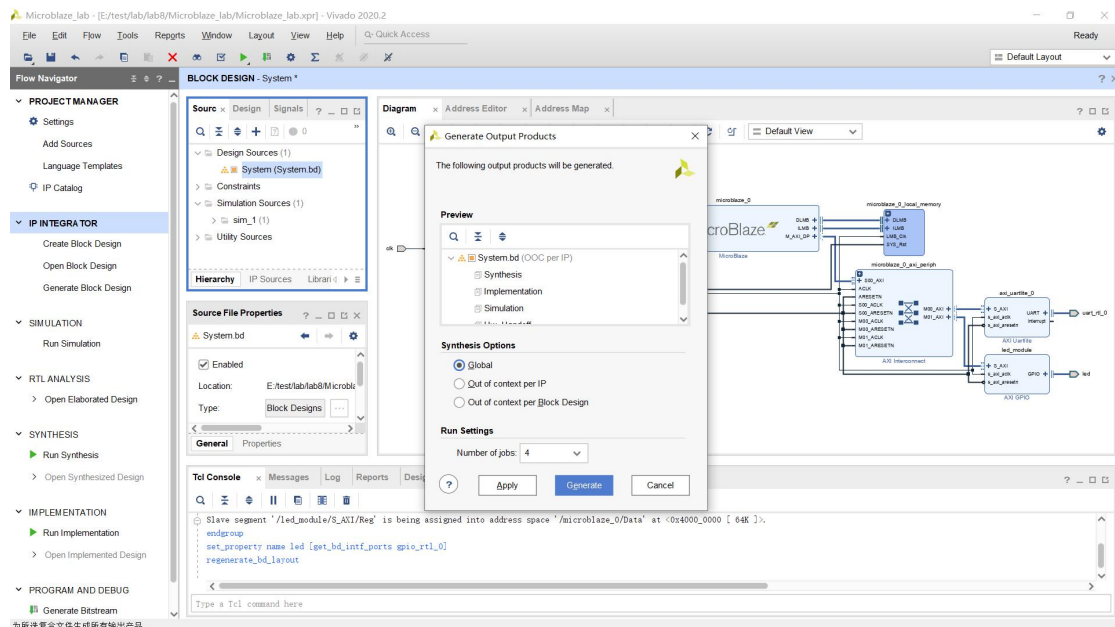
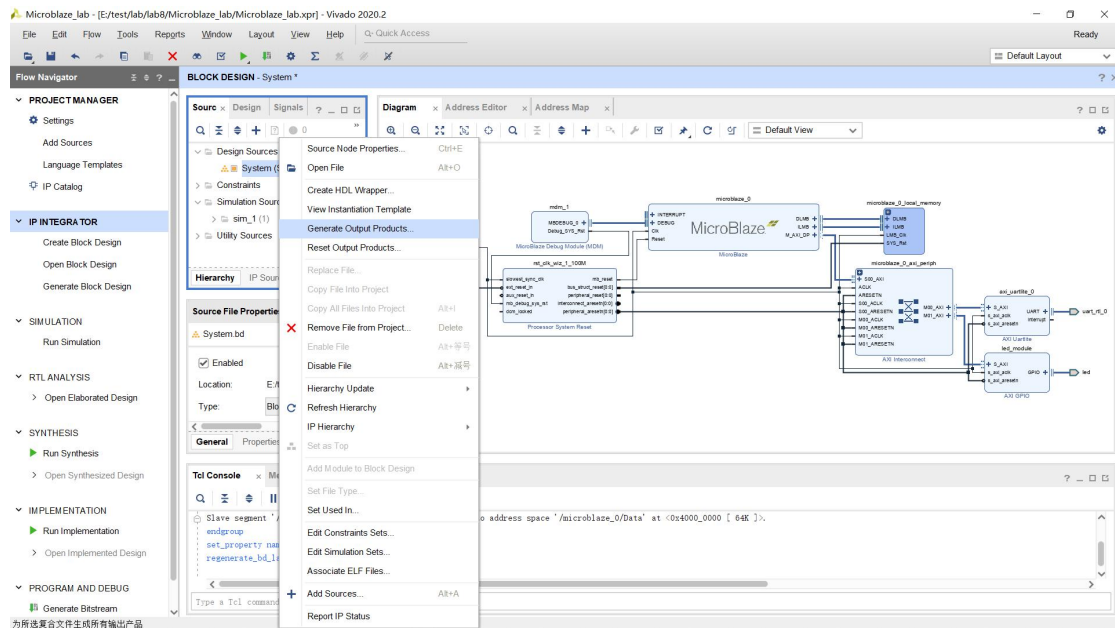


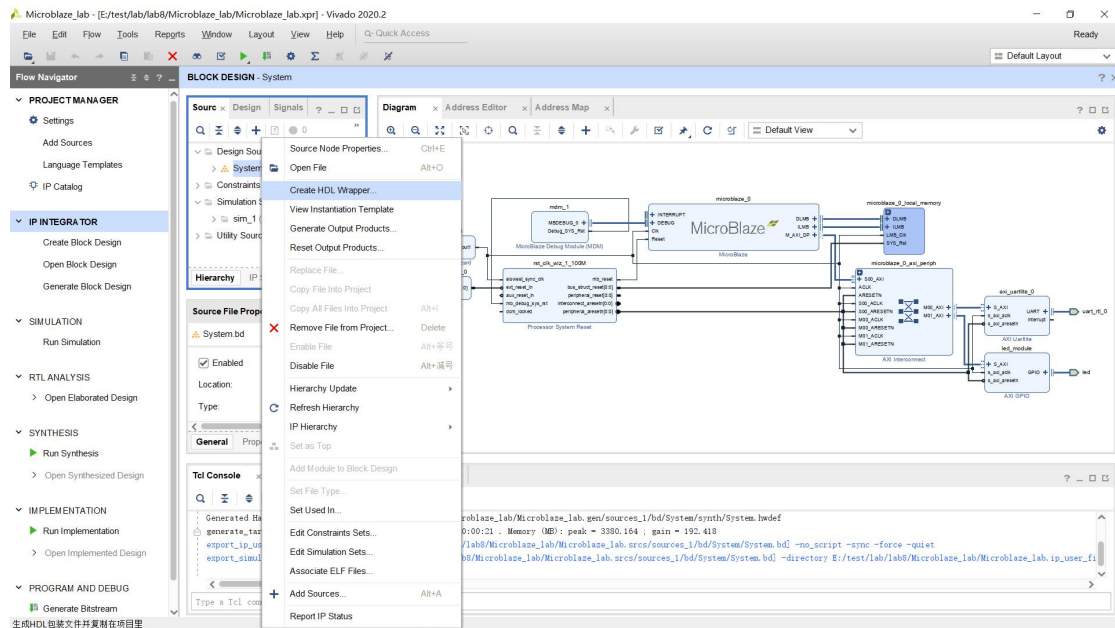


点击后的效果。

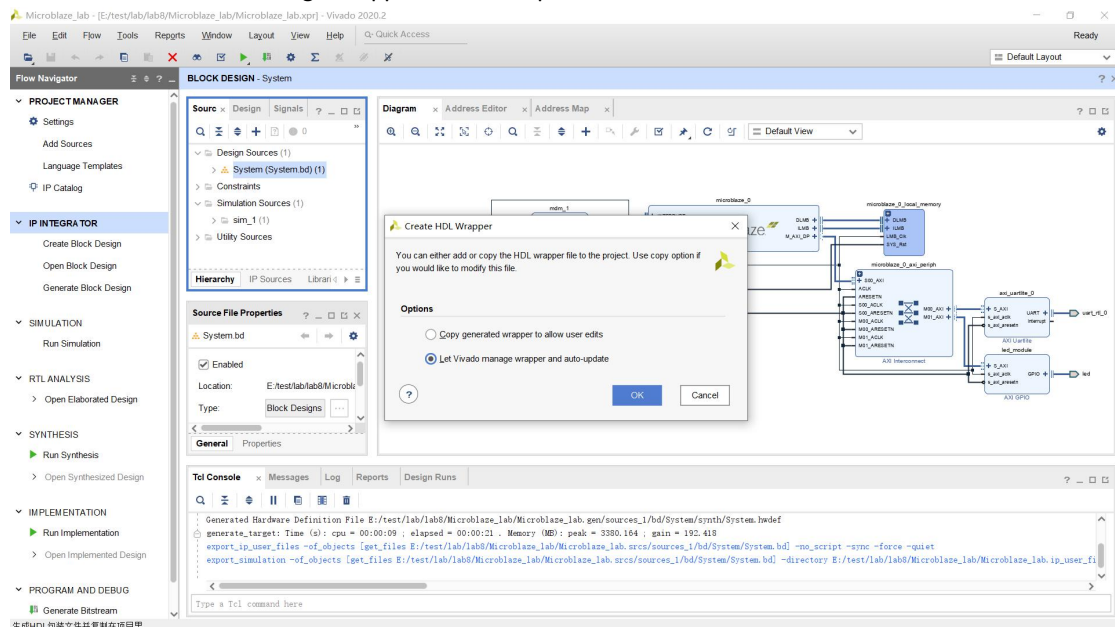


29. 选择 Sources 窗口的 System，点击右键，选择 Generate Output Products，生成相关 IP 的网表、设计、仿真文件。

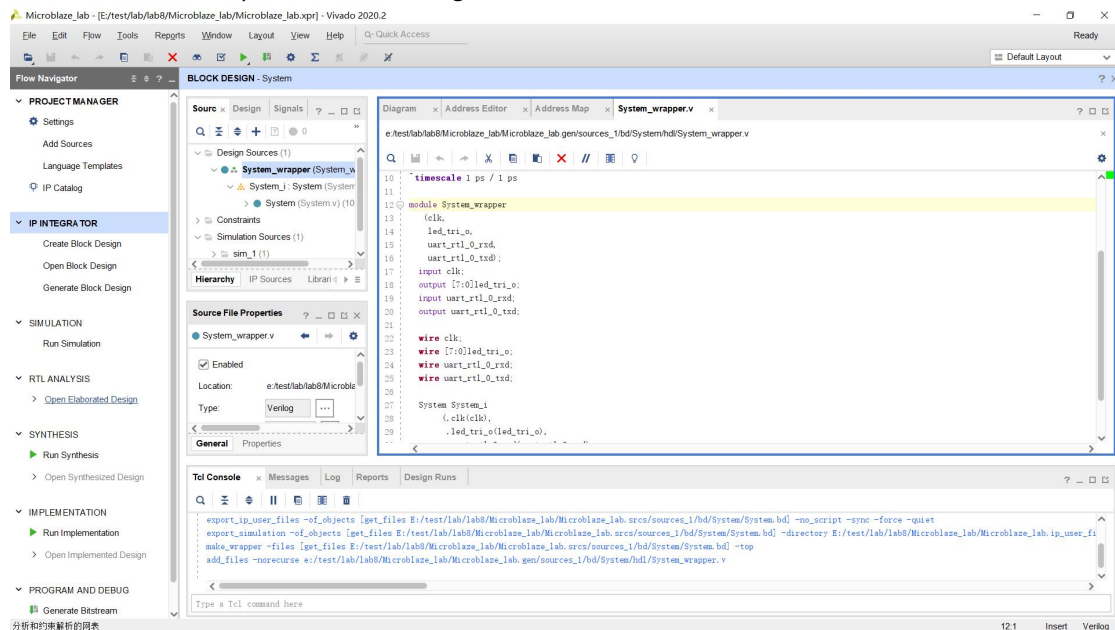
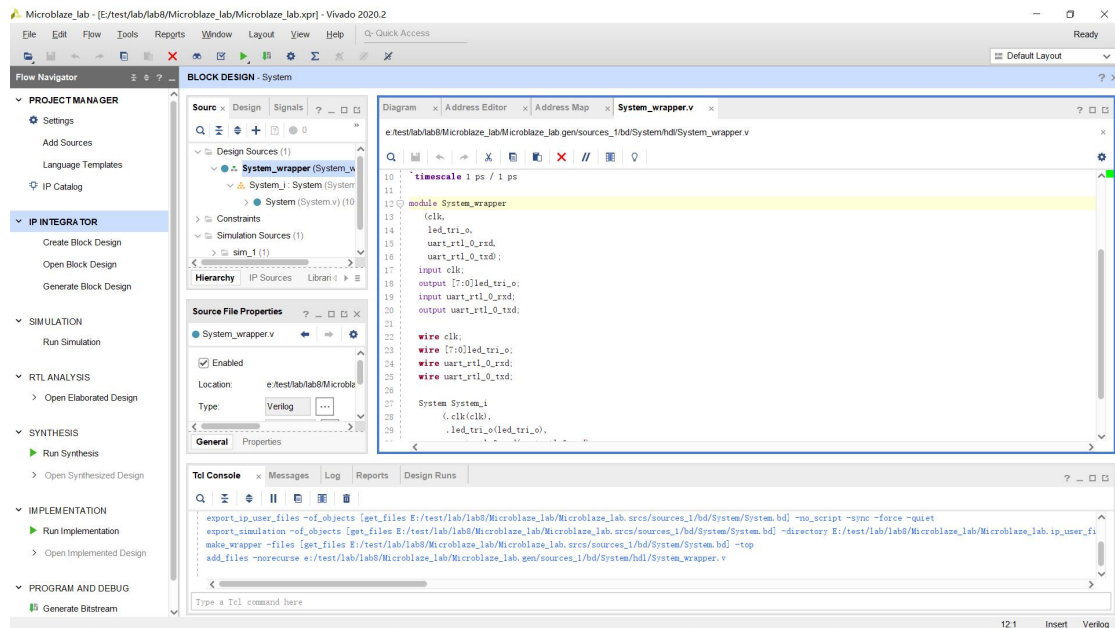


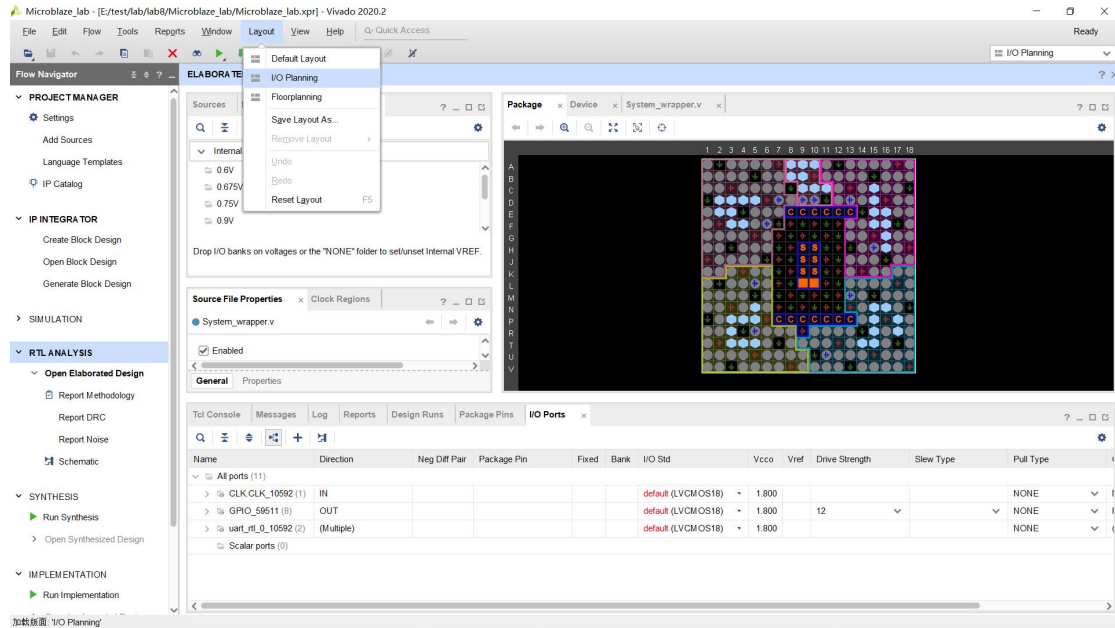


选择 Let Vivado manage wrapper and auto-update, 点击 OK。

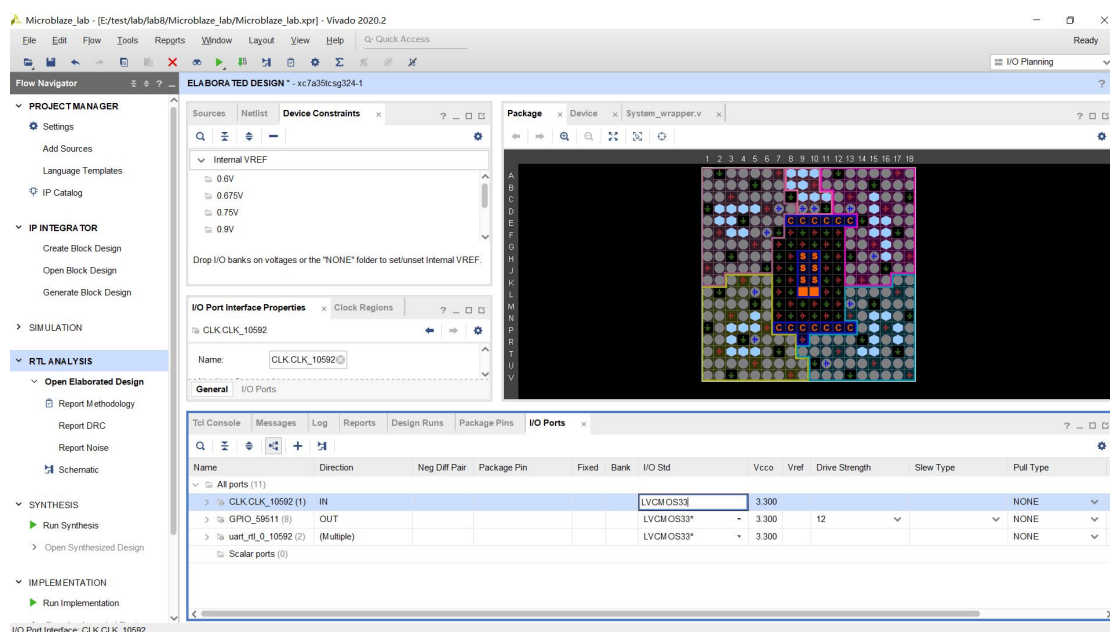


软件自动产生顶层 HDL 代码。

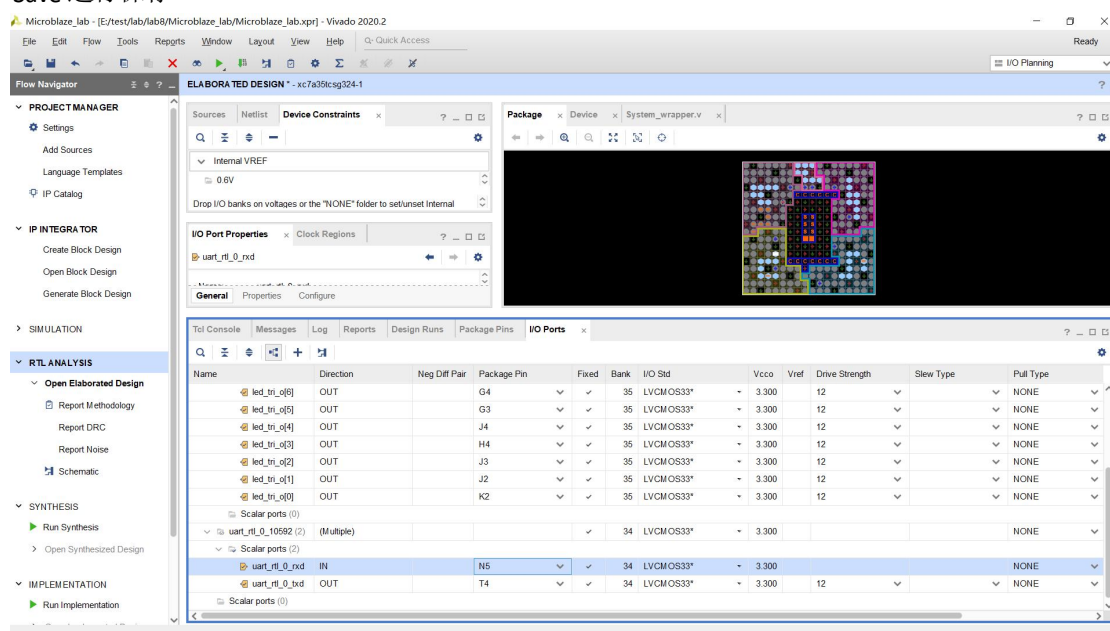




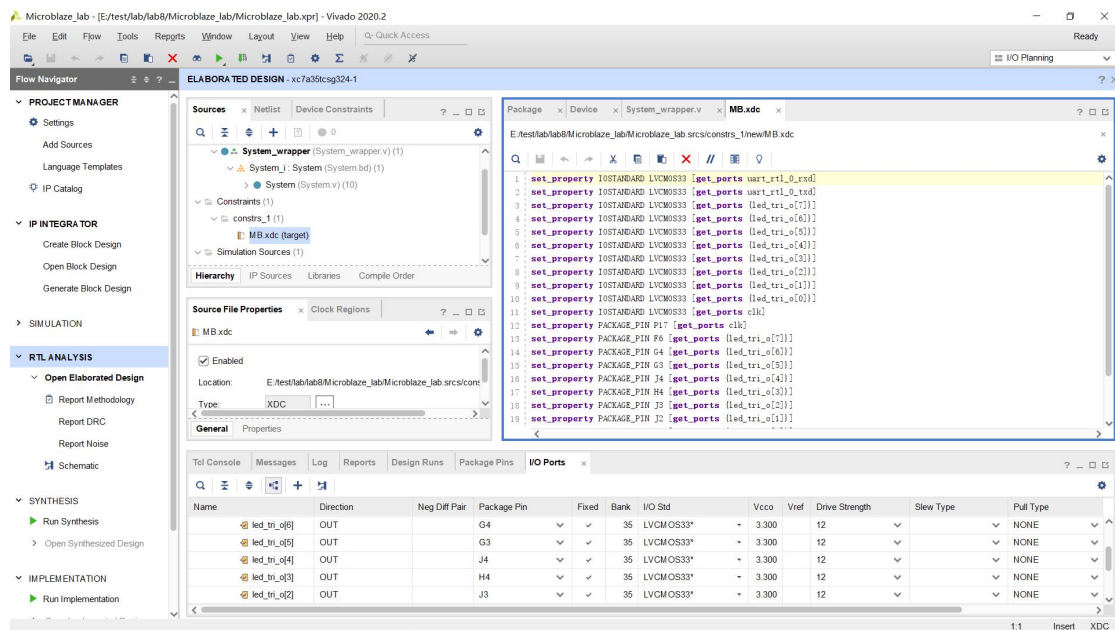
33. 把 I/O Ports 窗口里面所有电平都设成 LVCMOS33。



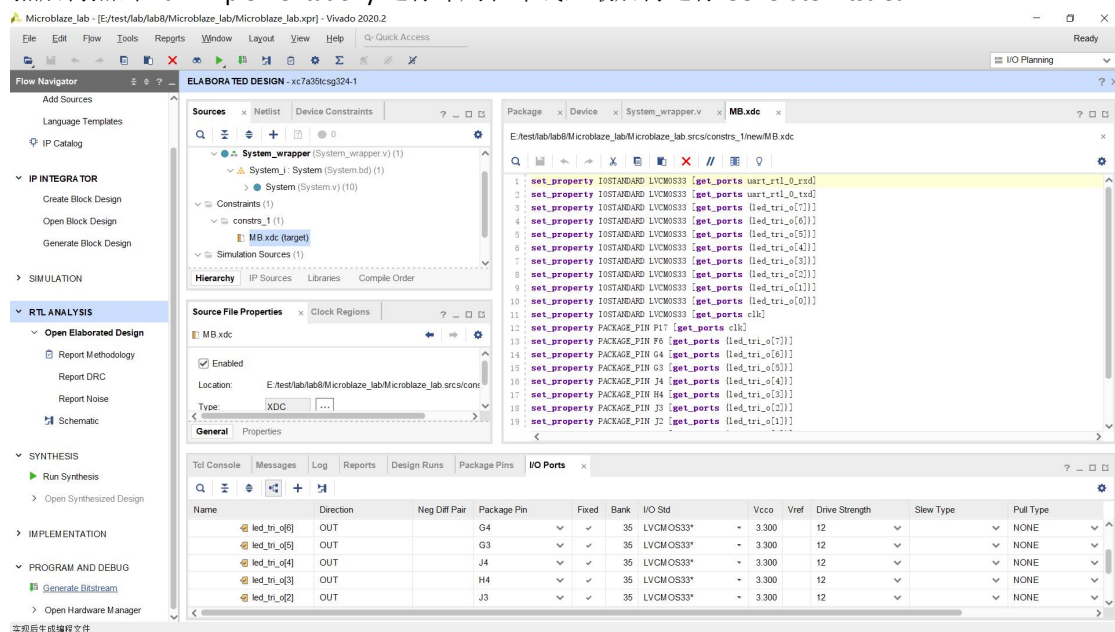
34. 并在 Package Pin 中输入绑定的管脚，在此我以 E-elements EGo1 开发板为例。点击 Save 进行保存。



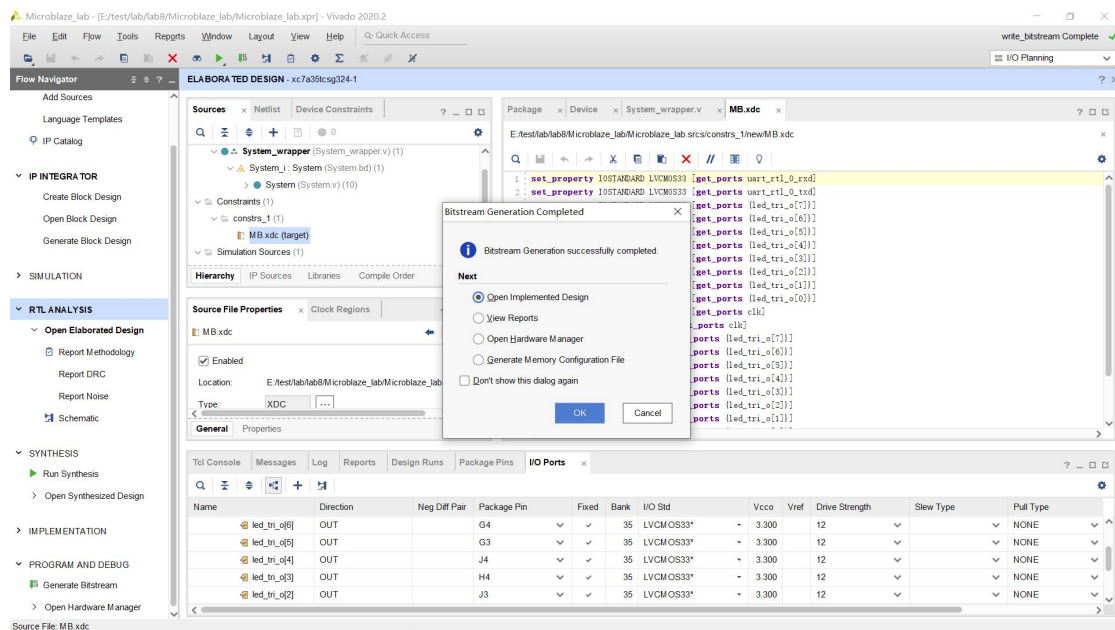
保存完后在源文件窗口 Constraints 栏目会多出一个生成好的约束源文件。



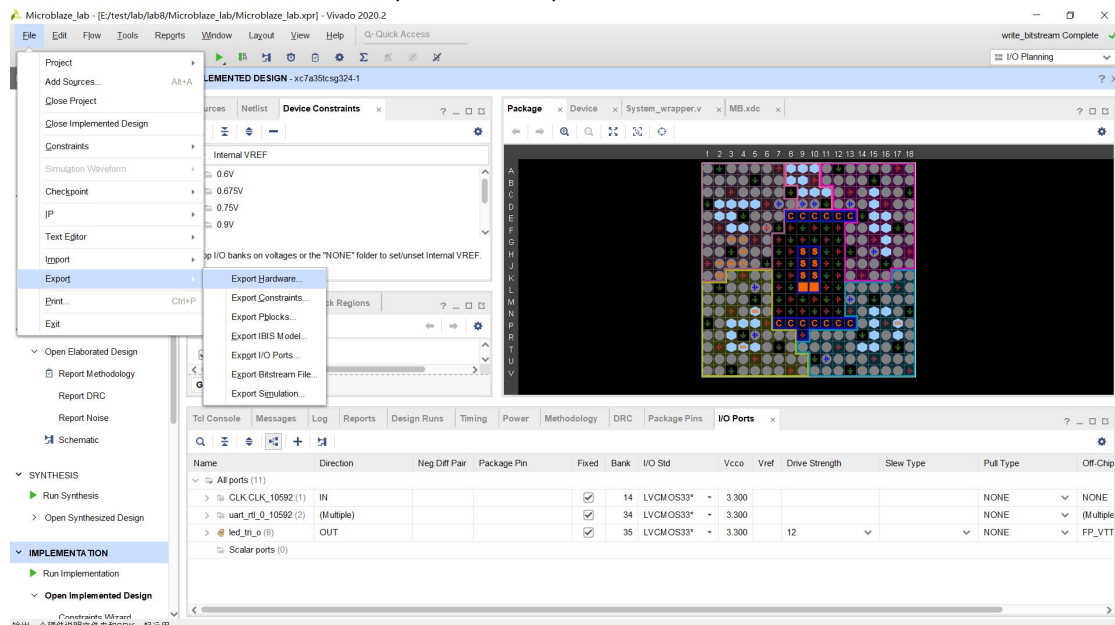
35. 可以直接点击 **Generate Bitstream**，软件会自动完成综合和布局布线，并生成硬件配置文件（如下蓝色框），亦可一步一步点击 **Run Synthesis**，让软件先对做好的工程进行综合。然后再点击 **Run Implementation** 运行布局和布线，最后再运行 **Generate Bitstream**。



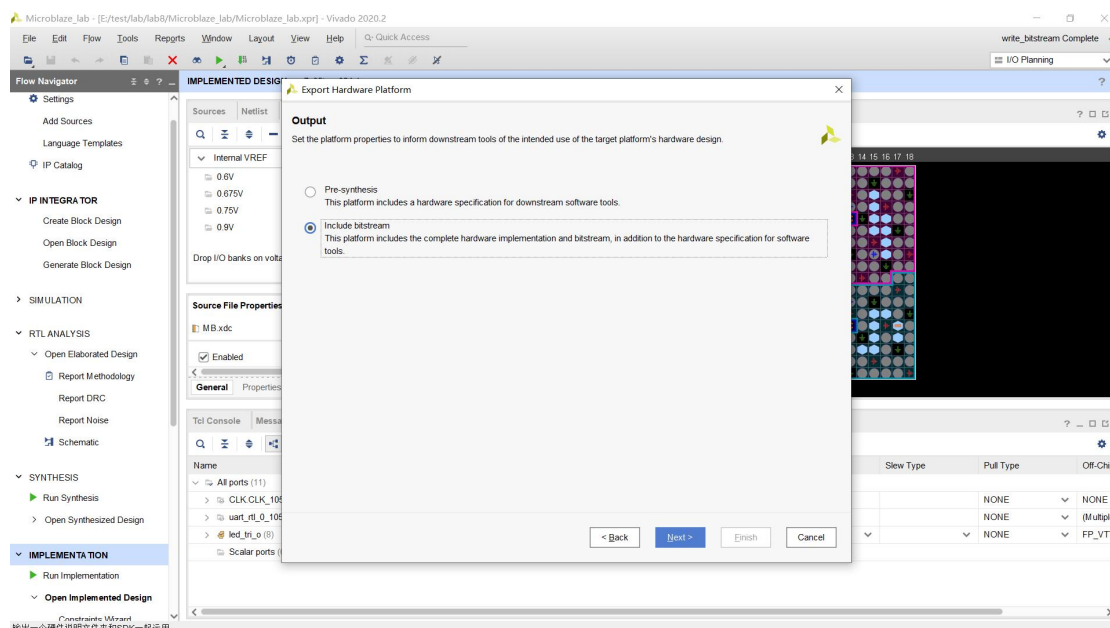
36. 当出现如下窗口，点击 **Open Implemented Design**，点击 **OK**。



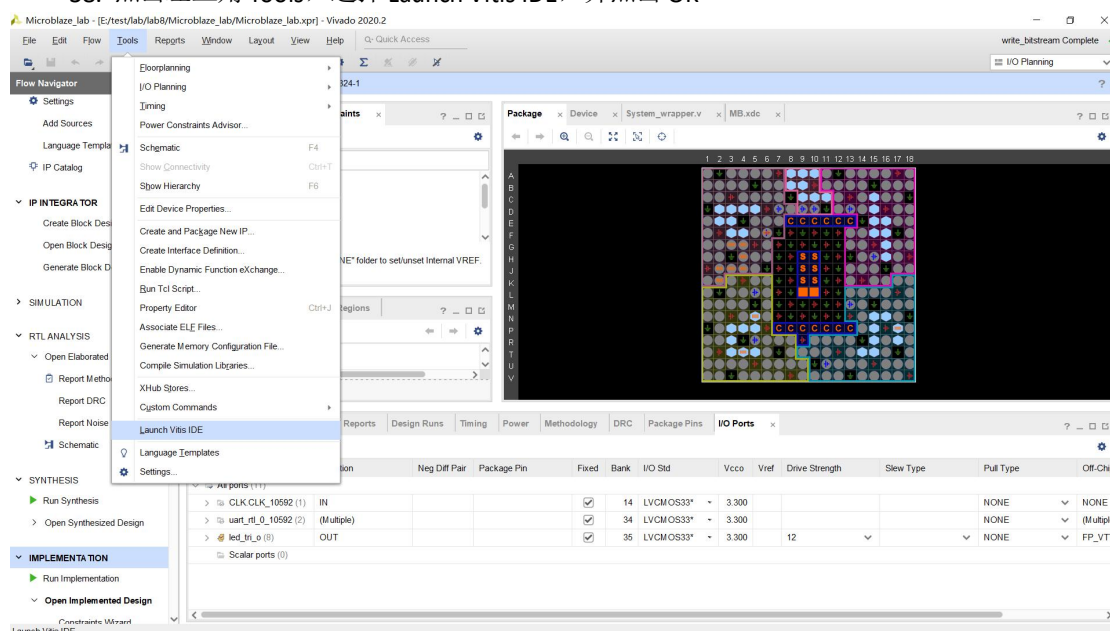
37. 点击左上角 File，选择 Export，选择 Export Hardware。



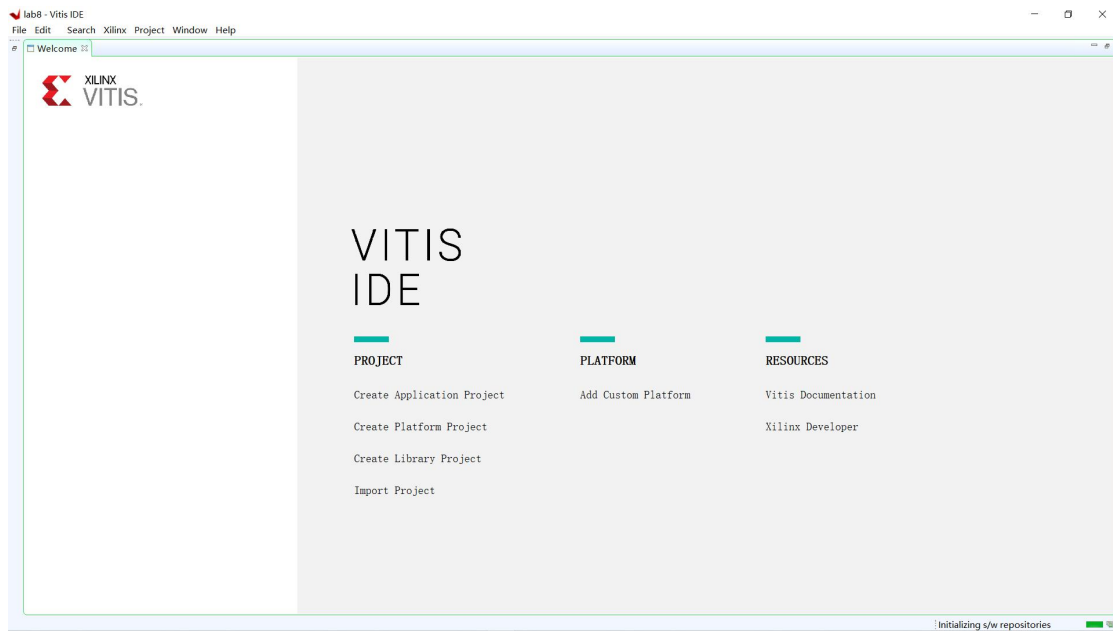
出现如下窗口，勾选 Include bitstream，并点击 OK。



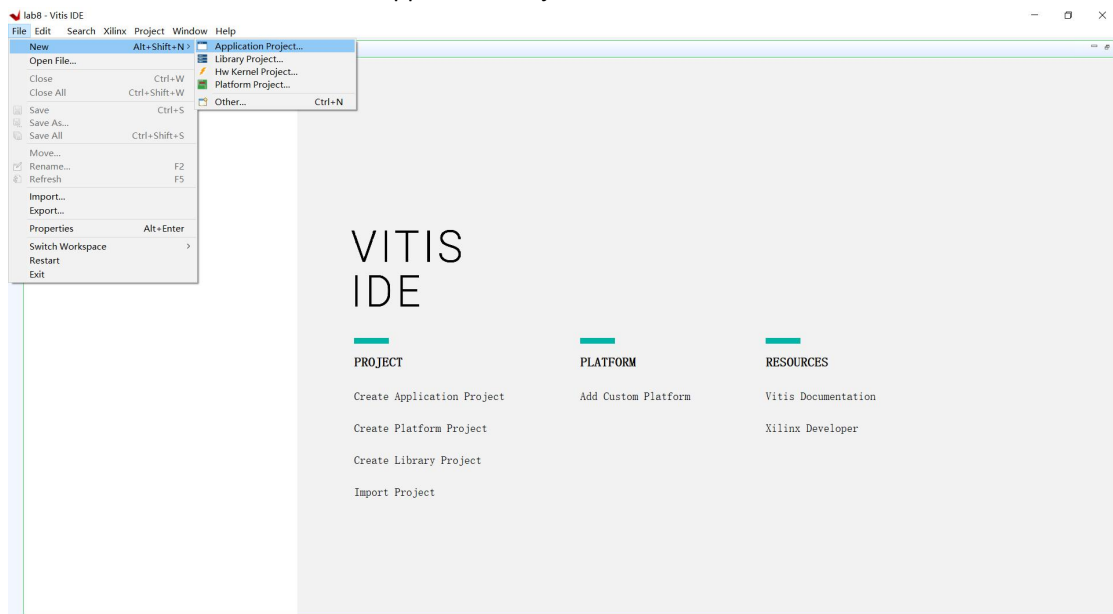
38. 点击左上角 Tools，选择 Launch Vitis IDE，并点击 OK。



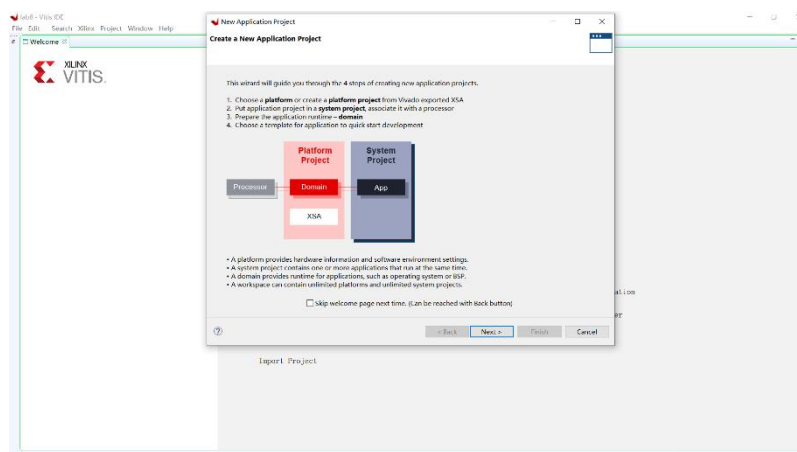
39. 软件自动打开 Xilinx Vitis 工具，用户可针对建立好的 MicroBlaze 工程进行软件编程，并会出现如下界面：

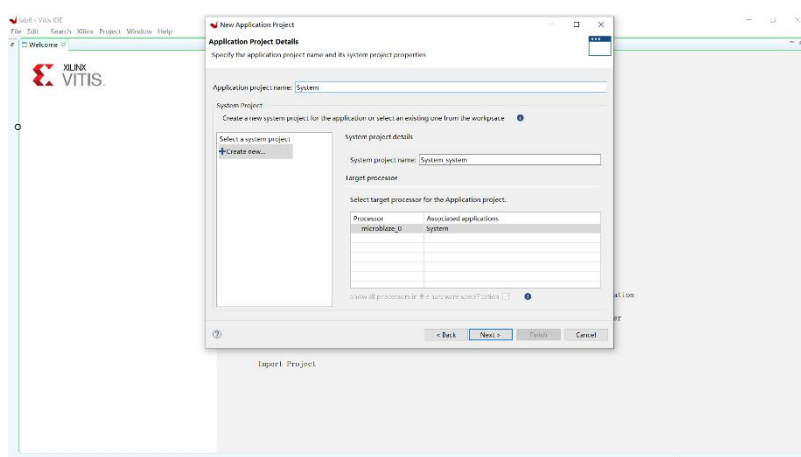
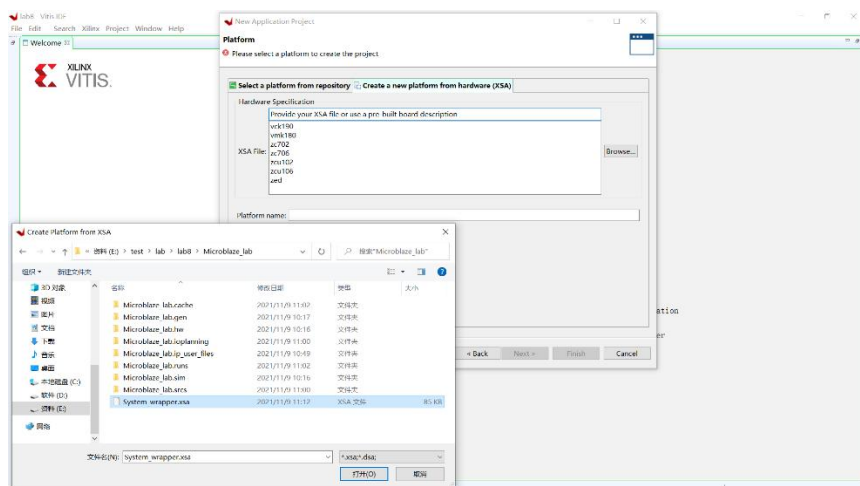


40. 点击左上角 File->New->Application Project。

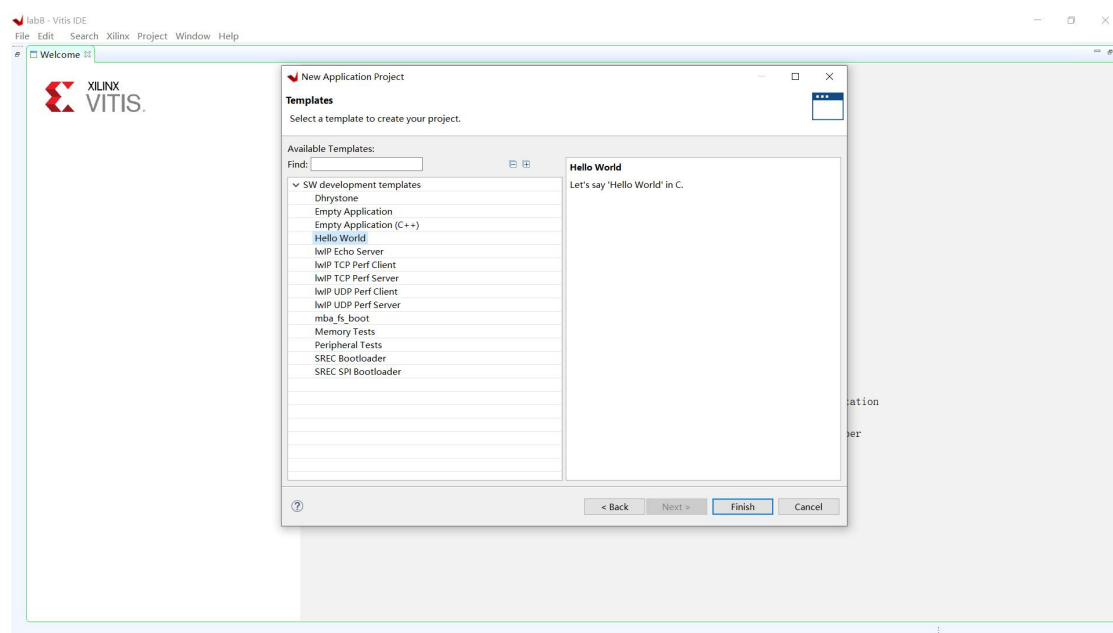


41. 出现如下界面，点击下一步，点 XSA->Browse->选择 Vivado2020.2 生成的 System_wrapper.xsa，之后输入工程名称。

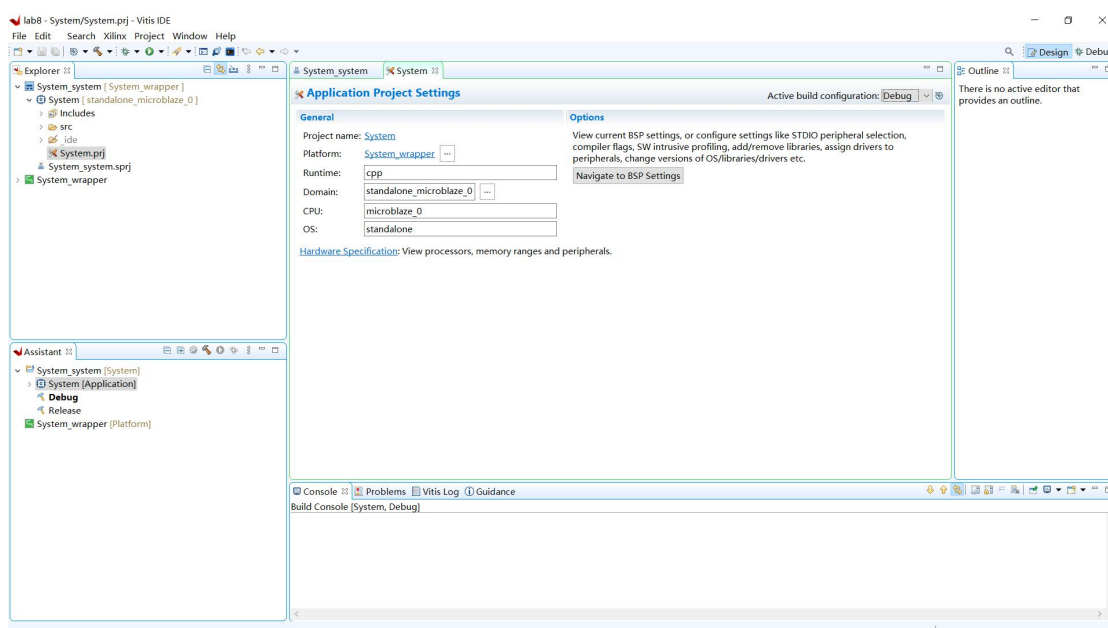




42. 选择源文件模板，可以选择空的原始应用，本次实验选择 Hello world 进行修改。点击 Finish。



43. 生成完成如下：



44. 关于 GPIO 的使用如下:

1) Initialize the GPIO

`XGpio_Initialize (XGpio *InstancePtr, u16 DeviceId)`

InstancePtr 是一个指针，指向一个GPIO实例，内存指针引用必须预先分配给调用方，Further calls to manipulate the component through the XGpio API must be made with this pointer.

DeviceId 是被Xgpio组件控制的唯一设备ID,由调用者或应用开发者决定通过在一个设备ID关联的通用xgpio实例到一个特定的设备

2) Set data direction

`XGpio_SetDataDirection (XGpio * InstancePtr, unsigned Channel, u32 DirectionMask)`

InstancePtr is a pointer to the XGpio instance to be worked on.

Channel contains the channel of the GPIO (1 or 2) to operate on.

DirectionMask 是一位掩码指定位输入，输出。设置为0位的输出和位设置为1的输入。

3) Read the data

`XGpio_DiscreteRead(XGpio *InstancePtr, unsigned channel)`

InstancePtr is a pointer to the XGpio instance to be worked on.

Channel contains the channel of the GPIO (1 or 2) to operate on

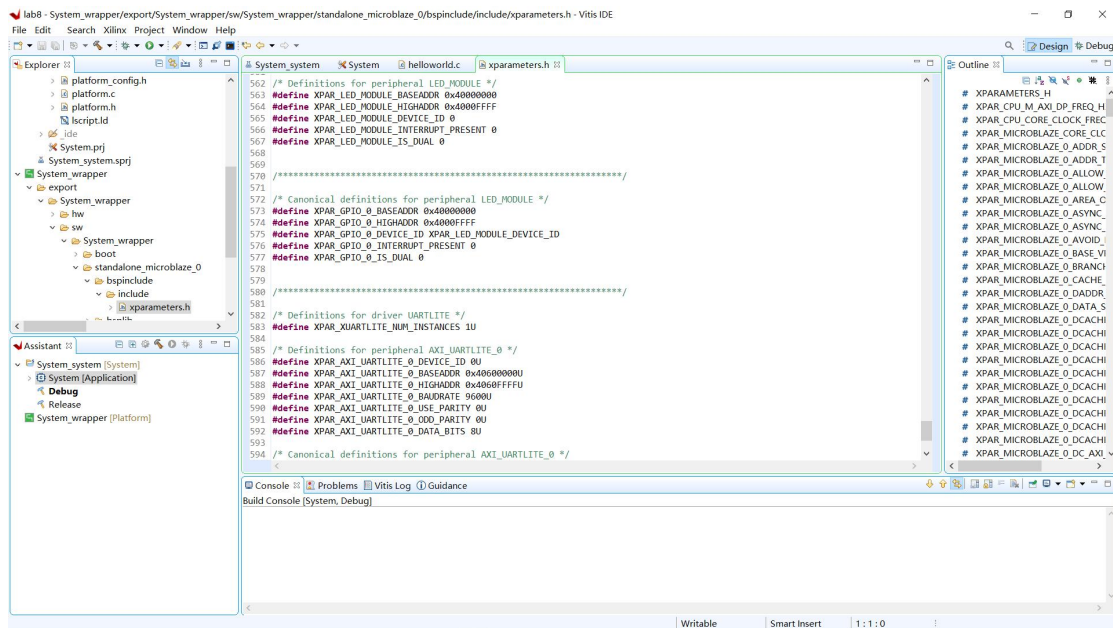
45. 双击 helloworld.c, 修改里面的代码为如下:

```
#include "xparameters.h"
#include "xgpio.h"
#include "xuartlite.h"

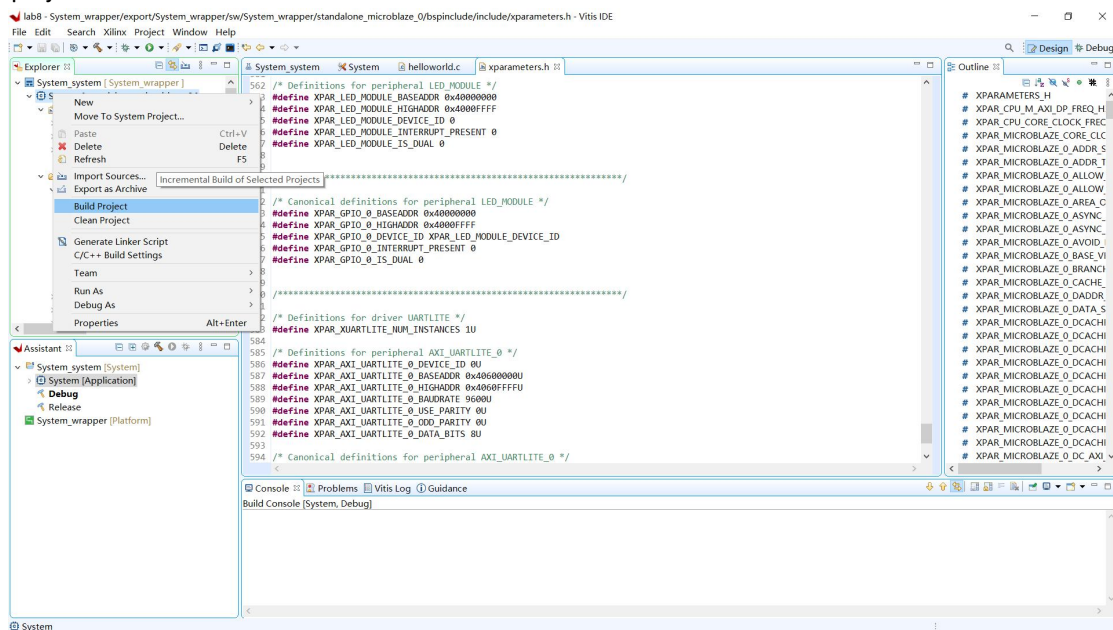
#define TEST_BUFFER_SIZE 16
u8 SendBuffer[TEST_BUFFER_SIZE]; /* Buffer for Transmitting Data */
u8 RecvBuffer[TEST_BUFFER_SIZE]; /* Buffer for Receiving Data */

int main(void)
{
    XGpio led;
    XUartLite UartLite;
    u8 led_check;
    XGpio_Initialize(&led, XPAR_LED_MODULE_DEVICE_ID); // Modify this
    XGpio_SetDataDirection(&led, 1, 0x00000000);
    XUartLite_Initialize(&UartLite, XPAR_UARTLITE_0_DEVICE_ID);
    while(1)
    {
        XUartLite_Recv (&UartLite, RecvBuffer, TEST_BUFFER_SIZE);
        led_check=RecvBuffer[0];
        xil_printf("show led");
        XGpio_DiscreteWrite (&led, 1, led_check);
        // LED_IP_mWriteReg(XPAR_LED_MODULE_DEVICE_ID, 0, led_check);
    }
}
```

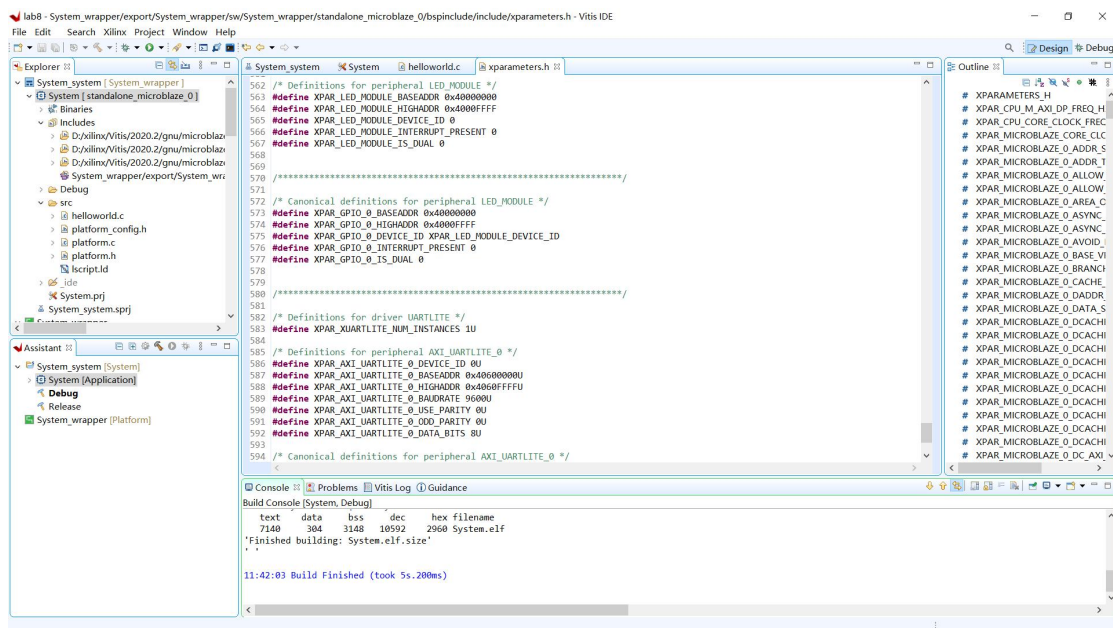
46. 代码里面的 XPAR_AXI_UARTLITE_0_DEVICE_ID 与 XPAR_UARTLITE_0_DEVICE_ID 可以在 xparameters.h 文件中查看。



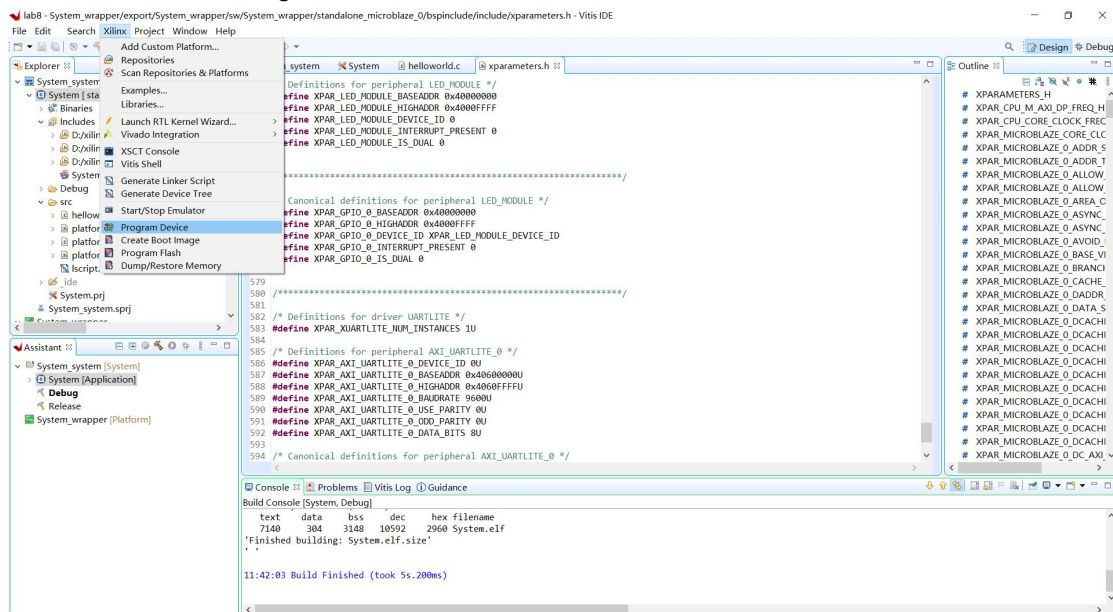
47. 点击保存，软件自动会对代码进行 build。或者对源文件打击右键，再点击 build project。



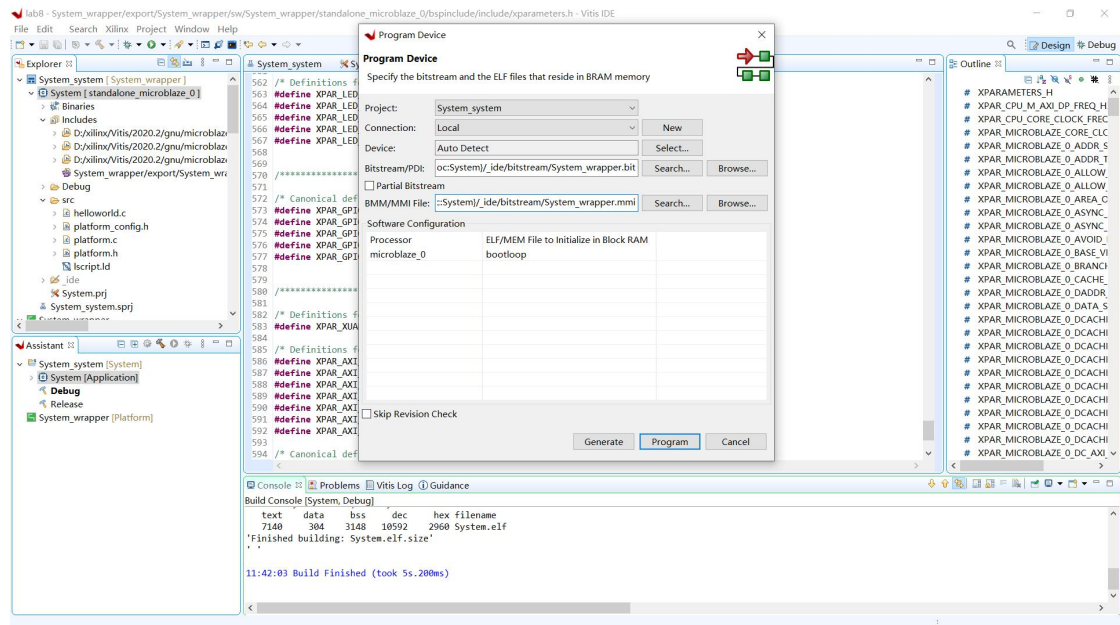
48. 当在 Console 中看到 Build Finished，即为成功。



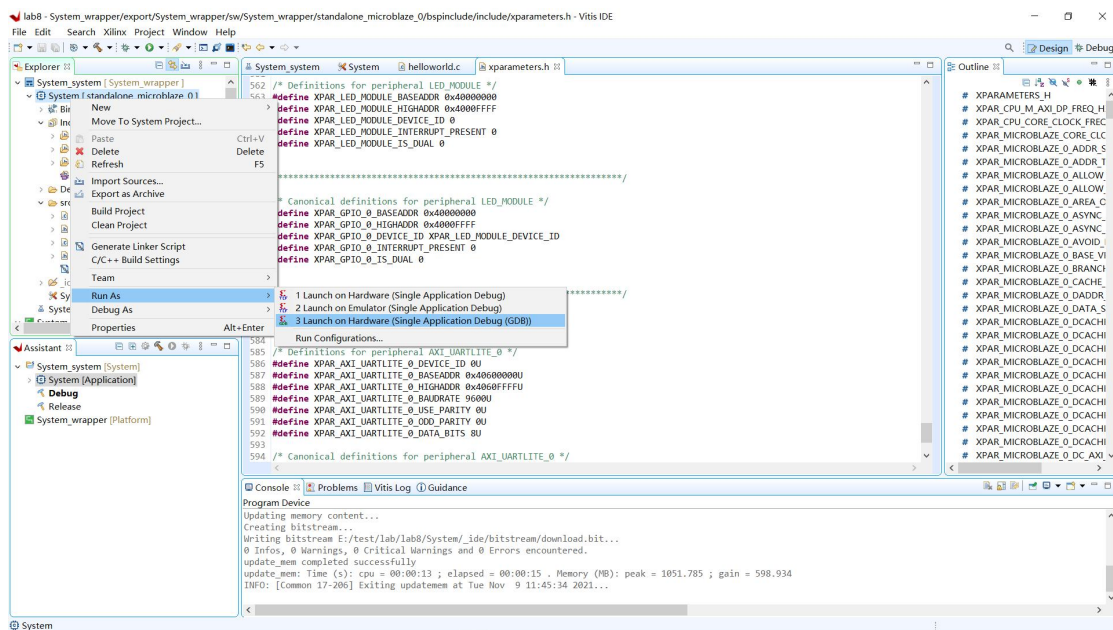
49. 选择 Xilinx->Program Device。



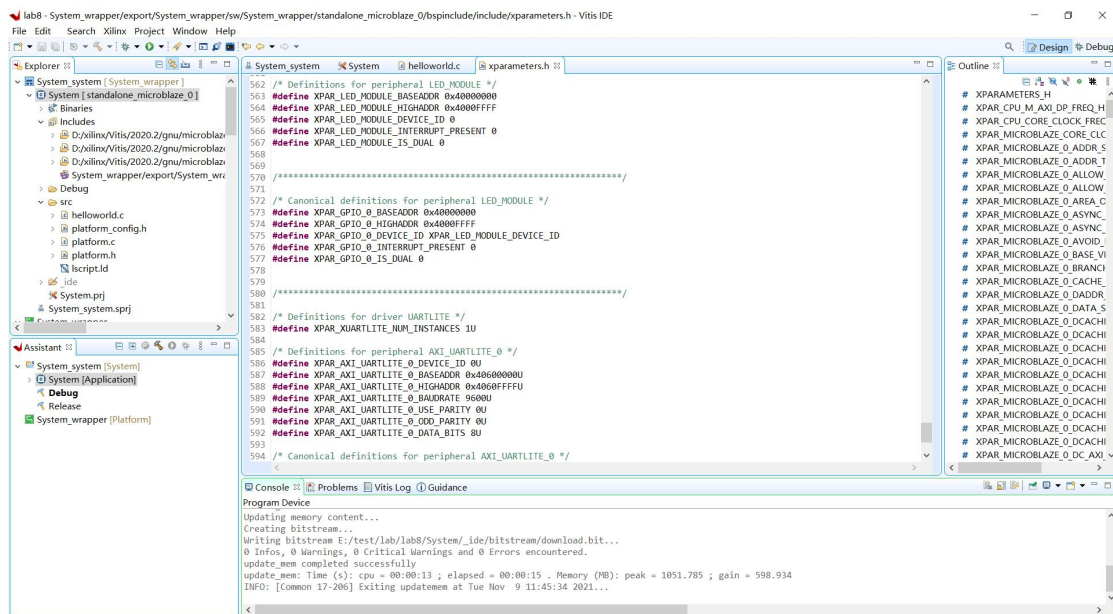
50. 软件自动找到本工程的.bit 和 MMI 文件，不能找到则点击 search 找到工程对应位置。点击 Program。



51. 在此选择源文件点击右键，选择 Run As->Launch on Hardware(GDB)。



52. 软件会把 elf 文件配置到 microblaze CPU。



53. 当完成后，打开任意串口调试助手，设置波特率为 115200。输入相应的值，LED 上会打印出 ASCII 码。

串口输入: 1



串口输入：2



串口输入：3

